COURSE AND RESEARCH PROGRAM

Machines of Understanding Through Circuits

SOP Lang, Sufficient Representations and the Controlled Construction of Reasoning

What must a representation preserve
for reasoning to continue?

Sînică Alboaie

Founding Intuition and Research Direction
Conceptual and Educational Development in Collaboration with ChatGPT

SEPTEMBER 2026

Contents

Foundations, Mechanisms, and Experimental Program

Contents / Continued

Before We Begin

What This Book Promises

A machine can flawlessly execute a program it could not have invented. It can verify a proof it would not have found. It can produce a fluent answer without being able to state, in a verifiable form, which relationships among facts support that answer. These three gaps—between execution and construction, verification and discovery, expression and justification—define the book's problem.

The starting point is an intuition of the human author, Sînică Alboaie: if any finite computation can be organized as a circuit of dependencies, perhaps understanding too can be approached as the construction of circuits. Such a circuit receives a representation of a situation and produces justified findings, predictions, or actions. The decisive question is no longer merely how we execute the circuit, but how we construct and extend it, and how we know that intermediate representations have not lost precisely the information we need.

SOP Lang is the research direction in which this intuition is to become a common language for knowledge and computation. The book does not assume that the language has already solved understanding. It builds the foundations needed to formulate a serious attempt and recognize an informative failure. It presents a proposed conceptual architecture, not certification of an existing implementation.

The contribution of the AI partner, ChatGPT, consists here in articulating the intuition, identifying mathematical precedents, developing examples, and formulating testable hypotheses. The AI partner's “acceptance” of an idea means that the arguments presented support it under the stated conditions. It is not independent evidence, scientific consensus, or validation of every consequence the human author may have in mind. The detailed proposals below remain open to criticism from both partners and future researchers.

How to Read This Book

No knowledge of category theory, mathematical logic, or abstract interpretation is required. Familiarity with variables, functions, conditions, tables, and program execution is sufficient. Additional concepts are introduced through small situations before receiving a formal name. Short formulas serve to pin down the meaning of explanations, not replace them.

The first part clarifies the question. The second builds the mathematical vocabulary. The third explains where combinatorial explosion comes from and when it can be compressed. The fourth applies the ideas to language and distributed representations. The fifth proposes a conceptual organization for SOP Lang. The final part turns promises into experiments, abandonment criteria, and directions for further work.

Readers in a hurry can begin with the chapters on sufficient interfaces, controlled observations, and the experimental program. Nevertheless, understanding the difference between a sound approximation and an exact equivalence is indispensable: almost every excessive promise in this area begins by confusing them.

Five Kinds of Claim

A philosophical intuition proposes a perspective on understanding or reality. A technical result has premises and a provable conclusion. An engineering decision chooses a trade-off. An observation describes what happened in a controlled model. A research hypothesis states what we should observe under conditions not yet explored. The book separates these within its argument, even when it does not repeat the label in every paragraph.

The numerical observations come from finite models that were rerun and checked. They are not findings about natural language, the brain, or the performance of a general system. The cited literature provides precedents and tools; the teaching examples and integration proposals are developed here. The bibliography is annotated for guidance, but the main explanations do not require consulting it to be intelligible. The references were checked on September 11, 2026.

Working thesis: a machine of understanding must learn not only which operations to execute, but also which distinctions are worth preserving for what might follow.

PART I

The Intuition and Its Limits

What we accept about computation, what we propose about understanding, and what does not follow about the nature of reality.

1. From an Answer to the Circuit That Justifies It

A Program That Answers and One That Understands Something

Consider a simple situation: a warehouse contains ten boxes; three are reserved; two of those reserved have already been shipped. The question “how many boxes are available?” cannot be answered correctly merely by identifying the numbers and performing a subtraction. We must establish what “contains” means, whether ten describes the stock before or after shipment, and whether a shipped box can still appear among those reserved. A single sentence conceals a small model of the world.

A numerical answer may be correct by chance. An explicit circuit should show which states were distinguished, which temporal relationships were accepted, and which definition of availability was used. When the description is insufficient, the legitimate result may be a question or two conditional answers. A machine that recognizes exactly what it does not know may demonstrate more operational understanding than one that immediately produces a number.

The circuit intuition begins with this fact: between input and answer there are transformations. Some recognize objects; others connect a referring expression to an object; others apply a rule; others choose a formulation. If the transformations and their dependencies become explicit, we can inspect the answer's mechanism and change a premise without blindly rebuilding everything.

This does not mean that every mental process already reduces to operations we know how to write. It means that we can adopt an engineering objective: to build systems whose useful results are supported by computational structures and justifications open to verification.

Why the Circuit Can Be Much Larger Than the Answer

The answer “Maria” may require identifying two people, recognizing a transfer, ordering events, resolving an ambiguity, and applying a persistence rule. The size of the answer does not tell us how difficult it was to construct. Nor does the simplicity of the rules guarantee that finding their combination will be simple.

Moreover, the final circuit often hides discarded alternatives. A system may have explored millions of possibilities to produce a final path of ten operations. If we publish only the path, we show the justification for the result, but not the cost of discovery. A serious research program must measure both objects: the accepted circuit and the process that found it.

In this book, “construction” includes choosing operations, arguments, representations, and hypotheses. Sometimes it means selecting an existing circuit. At other times it means composing fragments. In difficult situations, a new representation must be invented in which the problem becomes easier. Applying a known subtraction is not the same as discovering that an entire paragraph can be represented as a quantity-conservation problem.

The Human Intuition and the AI Partner's Proposed Reading

The strength of the human intuition does not lie in the simple assertion that programs have dependencies. It consists in bringing together three questions: can knowledge be expressed through the same structures as operations on it; can constructing a line of reasoning become an explicit computation; can disciplined intermediate values make this computation controllable?

The AI partner accepts that this is a coherent direction. It also accepts that intermediate representations can radically change search costs. It does not accept, without further conditions, the leap from “representable as a circuit” to “efficiently discoverable,” or from “auditable” to “true.” These reservations do not diminish the intuition's value; they turn it into a program with precise questions.

The proposition that will organize the rest of the book is this: if two different constructions contain exactly the same information for every relevant continuation, they need not be explored separately.

The difficulty is recognizing this situation without executing every possible continuation. The rest of the theory attempts to make this recognition local, compositional, and, in certain classes of problems, inexpensive.

What We Want to Achieve with SOP Lang

SOP Lang is treated here as a language of semantic circuits: representations in which the production and use of values are explicitly connected. A reusable rule, the interpretation of a sentence, a query over facts, and a composition strategy can share the same general circuit form. The uniformity sought concerns representation, not an assumption that all operations are equally reliable or equally well understood.

The aim is not to replace every neural component with symbols as a matter of principle. It is to distinguish what one component proposed, what another verified, what information was preserved, and what was approximated. If a vector representation is useful for selecting a circuit, it can be used. If the final answer must support an exact claim, selection must be followed by appropriate verification.

A modest but real success would be a system that answers within a narrow domain, recognizes ambiguity, produces verifiable justifications, and corrects itself locally when new information appears. A more ambitious success would be automatically learning representations that enable this behavior in new domains. The book does not confuse the two levels.

2. What We Mean by Understanding and What We Do Not Claim About Existence

Four Commitments That Must Be Kept Separate

The first commitment is methodological: we seek explicit models of dependencies among pieces of information. The second is testable through engineering: we want behavior robust to rephrasing, changes in premises, and new questions. The third is philosophical: we suspect that understanding involves identifying stable structures within variations in experience. The fourth would be ontological: reality itself is a circuit. The first three do not demonstrate the fourth.

A map can represent spatial relationships without the city being made of the map. Similarly, a computational description can fit a phenomenon without exhausting its nature. The proposed research need not establish whether the universe is discrete, whether the mind is entirely computable, or whether subjective experience can be reproduced. We must specify which behaviors we want to achieve and what would count as an explanation of them.

This boundary avoids two extremes. The first declares success before building anything, invoking the universality of computation. The second rejects the project because it does not solve every problem in the philosophy of mind. Neither helps the experiment. We can investigate competencies of understanding without claiming to have solved consciousness.

An Operational Definition, Not a Final Essence

We will say that a system exhibits a competence of understanding in a domain when it can use a representation of the situation to answer a family of questions coherently, anticipate consequences, identify missing information, and revise its conclusions when the premises change. The definition is deliberately relative to the domain and the tests. It is not a metaphysical verdict.

A test should not demand only the reproduction of a known answer. We can change the names, sentence order, a time interval, or a seemingly minor condition. We can ask why a conclusion does not follow. We can introduce an irrelevant fact and observe whether the answer changes unjustifiably. We can replace a premise and check whether the system withdraws precisely the conclusions that depend on it.

For example, “Ana lent Maria a book” and “Ana gave Maria a book to keep” may have the same immediate consequence for physical possession, but different consequences for ownership and the obligation to return it. A representation adequate for “who has it now?” may be insufficient for “who may legitimately sell it?” Understanding more sometimes means preserving distinctions that the initial question did not require.

Meaning Needs a Place in the World

A circuit can be correct relative to a mistaken interpretation. If an ironic sentence is treated as a literal report, subsequent inferences may be valid yet inappropriate to the situation. If a sensor is faulty, the computation can process false data flawlessly. We must therefore distinguish truth within a model, fidelity in interpreting the text, and the model's adequacy to the world.

For SOP Lang, this suggests keeping a source fragment, its interpretation, and the interpretation's epistemic status separate. “The document states that the door is closed” is not identical to “the door is closed.” The first statement can be checked against the document; the second requires trust in the document or independent observation. Provenance allows this transition to be audited, but does not automatically make it legitimate.

There is no single scale of certainty that can replace all these distinctions. A confidence score for recognizing an expression is not the probability that the event described occurred. A formal proof is not the probability that its premise is true. A machine of understanding must represent what kind of uncertainty it has, not merely how much.

What Would Count as Progress and What Would Be Only an Impression

Progress would mean transferring a learned representation to previously unseen combinations, with justifications that withstand verification. It would mean reducing search without eliminating valid answers. It would mean introducing a new distinction precisely when a counterexample shows that the old representation was too impoverished.

An impression of progress can arise when the system produces convincing explanations after guessing the answer, when tests resemble training examples too closely, or when the cost of constructing a library is hidden in human labor. These situations do not make the system useless. But they must be described accurately: human assistance, memorization, post-hoc verification, and genuine generalization are different outcomes.

The book's working philosophical position is modestly realist: we assume there are regularities independent of the system's wishes and that errors can be discovered by confronting data and arguments. We do not assume that every regularity has a short description, that every relevant truth is decidable, or that every goal can be achieved through feasible computation. The program seeks islands of exploitable structure, then studies how far they can extend.

3. What a Circuit Is When Its Wires Carry Worlds

The Circuit as Dependency, Not an Electronic Diagram

A computational circuit is an organization of operations and the values on which they depend. An operation receives values and produces values. A wire indicates that a value produced in one place is used elsewhere. That value need not be a bit. It may be a number, a vector, a table, a set of possibilities, or a description of another circuit.

If a function adds two prices, its wires carry numbers. If an operation joins two tables by customer identifier, its wires carry relations. If an operation interprets a sentence, its output may be a set of event structures. The same word, “wire,” describes the logical dependency between operations, not the material from which memory is made.

This generality must be controlled. We can hide the entire problem in a node called “understand everything,” but then the circuit explains nothing. A useful description specifies the permitted operations, their contracts, and the relevant costs. The level of detail may vary, but a boundary must separate explained mechanisms from components still treated as oracles.

SSA: Stable Names for Results

In SSA form, each result name is assigned only once in the static representation. When the state changes, we use a new version. Instead of saying that the same stock is modified three times, we distinguish the initial stock, stock after reservation, and stock after shipment. A conclusion can thus indicate exactly which version it depends on. SSA is an established compiler technique; it is not, by itself, a theory of meaning. [1, 2]

Loops do not disappear through this convention. A program may repeat an operation an unknown number of times, and the static representation must describe control of that repetition and the selection of values arriving along different paths. Unrolling a finite execution produces a sequence of concrete versions. The unrolled circuit and the program capable of generating arbitrarily long executions are different objects.

We must also pay attention to effects. Two different names referring to the same mutable memory do not guarantee independence. For a clean conceptual model, effects on the environment are represented explicitly: the prior state, the authorized operation, and the resulting state. An immutable name does not magically make everything accessible through its value immutable.

Three Objects That Must Not Be Confused

A circuit source describes how computation can proceed. An executed instance applies that description to particular inputs. The content produced is the resulting value: for example, a table of people and events. We can retain all three, but they are not interchangeable.

A general rule about transferring possession is not the same as the episode in which Ana gives Maria the key. The episode is not the same as the sentence describing it. And the materialized fact “Maria possesses the key after the transfer” is not identical to the circuit that derived it. Keeping these differences makes it possible to reuse the rule, correct the episode, and rerun only the affected consequences.

In SOP Lang, circuit uniformity should mean that these objects can connect through clear interfaces. It should not mean mixing them all into an opaque object called knowledge. A circuit can produce another circuit as a value, but executing that produced circuit remains a separate operation with its own conditions.

Feedback, Time, and Open Processes

A conversational system does not receive all its inputs at the outset. New observations appear, goals change, and some operations consult the environment. We can represent each stage as a state transformation, but should not pretend to know the entire graph of the future in advance.

Organizing work into epochs is one possible choice: within an epoch, certain hypotheses and versions are fixed; a new observation opens a new epoch, in which some conclusions are retained and others withdrawn. This is an engineering proposal for auditing, not a universal condition of intelligence.

Representing an entire execution as a circuit can consume considerable memory. Research must measure what is worth retaining: every elementary operation, only relevant inferences, or a certificate that enables verification of the result. An enormous log is transparent in principle, but may remain useless to a human. Auditability is not achieved merely by refusing to delete anything.

4. Compositionality: Why the Whole Need Not Be a Complete Surprise

What We Know When We Know a Component

A programmer uses a library without reanalyzing all its instructions every time. They need a contract: which inputs are accepted, what result is promised, and what effects may occur. Compositionality is the idea that we can deduce properties of the whole from the properties of its components and how they are connected.

Suppose one operation converts degrees Celsius to Fahrenheit, and another compares the temperature with a threshold. For composition, the fact that both use numbers is not enough. We need to know the unit and the meaning of the threshold. An interface that says only “number” is correct as a storage type and insufficient as a semantic type.

The same happens in language. An event table may contain the time of reporting or the time of the event. A temporal operation that confuses them may be perfectly executable and conceptually wrong. Contracts must expose distinctions that affect the continuation.

The Intuition of Category Theory

Category theory provides an abstract vocabulary of composition. We can view value types as objects and transformations between them as arrows. If one transformation's output matches another's input, we

can compose them. Identity leaves the value unchanged. Associativity says that formally grouping a compatible sequence does not change its total transformation. Monoidal theories add a way to represent computations side by side, not just in sequence. [3]

For the engineering reader, the benefit is not replacing functions with solemn terminology. It is the obligation to state what structure is preserved when moving from one representation to another. If we translate an event circuit into a temporal-constraint circuit, that translation should respect the relevant composition. Otherwise, fragments translated correctly in isolation may together produce a different meaning.

An elementary example: a translation preserving only the duration of each road segment can calculate the total duration of a route if durations add up and no synchronization conditions apply. If scheduled ferries enter the picture, segment durations are no longer sufficient: arrival time must also be preserved. The failure is not in composition as an idea, but in the interface chosen for composition.

Text as Circuit Has Precedents

We do not propose that the expression “text as circuit” begins here. In DisCoCirc, Bob Coecke studies a compositional representation in which sentences can update meaning systems and text can be organized as a circuit. This precedent matters precisely because it connects textual structure with evolving representations, not merely with classifying isolated sentences. [4]

For SOP Lang, the additional question is how we automatically select and develop circuits, which interfaces allow alternatives to be merged, and how we verify that compression does not eliminate a necessary distinction. These objectives must be compared with existing work, not presented as new consequences of a mere notation.

What Compositionality Does Not Solve

A set of perfectly compatible pieces can admit an enormous number of constructions. Knowing what happens after connecting two components does not tell us which two components should be connected. Composition theory provides a foundation for verification and organization; the search algorithm needs additional structure.

There is another limit: contextual meaning may depend on things outside the selected components. The expression “it is warm” may be a description, a request to open the window, or irony. A model can treat this context as an additional input. But it cannot eliminate it merely because it wants elegant composition.

The working principle thus becomes precise: when composition fails semantically, we ask whether the operation is wrong, the interface too impoverished, or the context missing. These three explanations require different repairs. This diagnosis is already a practical contribution of the circuit perspective.

PART II

The Mathematics of Partial Information

A vocabulary built from scratch: possibilities, relations, refinement, and sound descriptions of states.

5. Sets, Relations, and Questions About Possible Worlds

An Unknown Value Is Not the Absence of All Information

An ordinary program often receives an exact value: the temperature is 18. In reasoning, we frequently have something else: the temperature is between 15 and 20; the person who entered is Ana or Maria; the event happened after lunch but before the meeting. We can compute with such information without prematurely inventing an exact value.

Here, a set is simply a collection of possibilities. If a wire can take the values 2, 3, or 4, we can describe it by the set {2, 3, 4}. New information, “the value is even,” leaves {2, 4}. We have not yet chosen the value, but we have made progress. The empty set shows that the simultaneously imposed information cannot be satisfied.

This interpretation is intuitive, and dangerous if we forget what the possibilities contain. Sometimes they are individual values. At other times they are complete world states. If we say only that Ana or Maria entered and that Ana or Maria left, we do not know whether it was the same person. Two individual sets can hide a decisive correlation.

Functions Transform; Relations Permit

A deterministic function associates an output with each permitted input. A relation describes permitted combinations. The equation x + y = z can be used to calculate z when we know x and y, but also to restrict x when we know something about y and z. Conceptually, the relation does not privilege a single direction of use.

Suppose we know that y is between 7 and 9, z between 10 and 12, and x between 0 and 100. The relation implies that x must be between 1 and 5. We have not inverted a function at an exact value; we have propagated a necessary condition. If we also learn that x is even, 2 and 4 remain. If we also learn z = 10, the constraints can be propagated again.

A relation can be stored explicitly as a table of combinations, but that is not the only option. The formula x + y = z is a compact description of many combinations. A table of a million equal pairs can be replaced with the relation x = y, provided the domain and meaning of equality are known. The choice of representation affects both memory and which operations are easy to perform.

"A Possibility Exists" Does Not Mean "The Conclusion Follows"

Suppose a text has two possible interpretations. In the first, Ana is at the station. In the second, Maria is at the station. The question "is it possible that Ana is at the station?" has an affirmative answer. The question "do we know that Ana is at the station?" does not have the same answer.

A conclusion is supported by all remaining possibilities only if it is true in every one of them. A conclusion is compatible with the information if it is true in at least one. These two criteria correspond to the intuitions "necessary within the model" and "still possible within the model." A system that confuses them may present a plausible hypothesis as a derived fact.

There is also the case of inconsistency: no possible world remains. In classical logic, certain definitions of implication make every proposition a consequence of incompatible premises. For a machine of understanding, operational policy must prevent presenting this phenomenon as success. The result should be marked "incompatible premises," with an explanation of the conflict, not "every answer has been proved."

Two Ways of Combining

When two constraints hold simultaneously, we intersect the possibilities. "x is between 0 and 10" together with "x is greater than 6" narrows the domain. When we have alternative hypotheses, we take the

union of the cases: “Ana left” or “Maria left.” Union does not authorize taking one effect from the first hypothesis and another from the second as though they coexisted.

For language, this rule is crucial. If Ana receives the key in one interpretation and Maria receives it in another, we cannot conclude that both people received it. We must preserve case identity, either explicitly or through a shared representation that preserves the dependencies. A smaller representation is useful only if it does not create nonexistent worlds.

That is why a “wire state” may be a small relational model, not just a value with a confidence percentage. When asking how to discipline SOP Lang outputs, one option is to require them to specify whether they represent simultaneous facts, alternatives, conditions, or exact results. A value's epistemic type matters as much as its data type.

An Intuition Exercise

Three boxes, A, B, and C, each contain an object. We know that exactly one contains a key. For each box considered separately, the possible values are “key” and “not a key.” If we retain only these three individual sets, we accidentally allow both the case with three keys and the case with no key. The essential information is the relation “exactly one.”

We need not enumerate every world to preserve this relation. We can retain a cardinality constraint. This is the first example of the general principle: a good representation is not necessarily the one containing fewer symbols, but the one in which the important relationship remains easy to use.

6. Information Orders, Refinement, and Fixed Points

More Information Means Fewer Possibilities

If we know that a number belongs to the interval [0, 100], then learn that it belongs to [20, 30], the second description is more informative. The set of possibilities has shrunk. We will call this process refinement. It must not be confused with the world changing: the number may have stayed the same while we learned more about it.

There are two common mathematical conventions. We can order descriptions by inclusion of their sets of possibilities: the small set is below the large set. Or we can order them by amount of information: the more precise description is above the vague one. These are opposite directions of the same idea. The book will state explicitly what is being narrowed, rather than hide the convention in symbols.

In the inclusion order, the universe of all possibilities represents lack of knowledge, and the empty set represents incompatibility. They are not the same. “We do not know which value is correct” allows many values. “No value satisfies the premises” allows none.

What a Lattice Structure Is

For our purposes, a lattice is a space of descriptions in which we can combine information systematically. For sets, intersection finds what satisfies two conditions simultaneously; union retains both families of cases. For intervals, intersection remains an interval or becomes empty. The union of two separate intervals is not always an interval, so a representation restricted to intervals may use the smallest interval containing both.

If the alternatives are x = 1 and x = 9, the interval [1, 9] preserves both values but also introduces 2, 3, and others. This is an approximation. It is not an error if we know the guarantee it preserves: it loses no real possibilities, but may include additional ones. Later, a candidate found among those additional possibilities must be checked.

The advantage of a disciplined structure is that we know how descriptions combine and what kind of loss may occur. We no longer let each operation arbitrarily invent an output format. Abstract interpretation uses such structures to analyze programs through approximate descriptions of their states. [5]

Information Propagation

Imagine three cells containing information about x, y, and z, connected by the relation x + y = z. When we learn something new about any of them, we can try to refine the others. A propagator network generalizes this idea: components transmit consequences, and cells combine incoming information. Radul and Sussman's work provides an explicit model of this organization. [6]

For a finite example, let the initial domains be x, y, z = {0, 1, 2}. We impose x < y and y < z. From the first relation we eliminate x = 2 and y = 0. From the second we eliminate y = 2 and z = 0. Returning to the first, y = 1 forces x = 0; returning to the second gives z = 2. A single pass was insufficient. Useful information circulated through the network.

The fixed point is the state in which another application of the propagators no longer changes the descriptions. It does not automatically mean that we have found every solution or know an exact value. It means that the propagation rules in use can add no more information within the available representation.

When Propagation Terminates

If domains are finite and every change eliminates at least one possibility, the number of strict changes is finite. If execution scheduling does not ignore a relevant propagator indefinitely, stabilization is reached. For monotone refinement operators applied fairly in a suitable finite setting, we can obtain the common closure determined by those operators.

This statement has important premises. If a component can arbitrarily add and remove the same information, it may oscillate. If it continually introduces new objects, the space is no longer finite. If some operations rely on the provisional absence of a fact, adding that fact later may require withdrawing a conclusion. We are then no longer dealing solely with monotone refinement within a fixed state.

In infinite domains, even a monotone sequence may continue without stopping. Loop analysis sometimes uses a controlled enlargement of the description, called widening, to force stabilization. The price is precision. It is not proof that the real process terminates, but a technique through which analysis produces an approximation within a controllable time. [5]

The Connection with SSA and Knowledge Revision

Refinement does not require destroying the old value. We can retain the initial description and create a new, more precise version linked to its justification. In an SSA trace, this organization shows which new

information produced each narrowing. For efficient execution, a system may use updates internally; the audit interface can expose versions. The concrete choice remains an engineering one.

Revision is different from refinement. If we learn that a premise was false, we do not necessarily narrow the old set of worlds; we replace it with another that may readmit previously eliminated possibilities. That is why the origin of every constraint must be preserved. Without provenance, the system does not know what to retract when a source is corrected.

For SOP Lang, this chapter suggests a general contract: an operation must state whether it produces a conclusion, refines a description, proposes an alternative, or revises a hypothesis. All can be circuits, but they must not be combined as though they had the same logical meaning.

7. Abstract Interpretation, Explained Through What We Choose to Forget

Why We Do Not Always Compute with Exact Values

An analyzer may want to know whether a program ever divides by zero. It is not interested in every possible numerical result, but in a relevant property. If the variable is definitely positive, certain checks become simpler. If it may be zero, the analysis retains a warning. The idea of abstract interpretation is to perform a computation about properties of values rather than enumerate all the values. [5]

Let us choose an extremely simple abstraction: parity. Numbers are summarized as “even” or “odd.” Adding two odd numbers gives an even number; adding an even and an odd number gives an odd number. For questions about the parity of a sum, the summary is sufficient. For “is the sum greater than a hundred?” it is not.

An abstraction, therefore, is not good or bad in absolute terms. It is suitable or unsuitable for a family of operations and questions. This relativity will become essential to circuit construction.

Two Bridges Between Concrete and Abstract

Let α denote the operation that transforms a concrete state into its abstract description. For example, α(7) = “odd.” Let γ denote the interpretation of an abstract description as a set of concrete possibilities. Then γ(“odd”) is the set of odd numbers. The letters hide no mysterious mechanism; they name the two directions of the connection.

A concrete operation F transforms values. An abstract operation F# transforms descriptions. For soundness, we require every possible concrete result to be included in the abstract description produced. In compact notation:

$F(\gamma(a)) \subseteq \gamma(F\#(a))$.

The formula says: if I start from any value permitted by description a and execute F, I do not end up outside the possibilities announced by F#. An analysis may announce too many possibilities, but when promising this guarantee it must not eliminate the real ones.

Let us use intervals. For x in [2, 5], the computation 2x + 1 falls within [5, 11]. Here the interval describes the actual results exactly if x varies continuously. For integer values, it also admits even numbers that do not occur; the representation may remain sound but less precise. The concrete domain matters.

Soundness Is Not Precision

If an analysis says “the result may be between 0 and 100,” and the actual result is 7, it has not erred in the sense above. It may nevertheless be useless for the objective. If the question is “is the result below 10?” the description does not decide it. A sound analysis may answer “I do not know” too often.

The soundest possible summary, “anything can happen,” is almost useless. That is why research must measure soundness and discriminating power together. In controlled cases, soundness can be checked exhaustively. Discriminating power can be measured through the number of conclusions decided, candidates correctly eliminated, and false alarms remaining.

An overly precise summary may cost as much as the concrete state. A representation preserving every possible world loses nothing, but may become exponential. The engineering question is not “can we preserve everything?” but “what structure lets us preserve exactly the important relationships at an acceptable cost?”

The Example in Which Intervals Forget Too Much

Let x be a number between 0 and 100, and let y = x. We want to compute x − y. In reality the result is always zero. If we retain only the individual intervals x in [0, 100] and y in [0, 100], interval subtraction produces [−100, 100]. The analysis is sound, but has forgotten the equality between the variables.

The repair need not involve enumerating the values of x. We can preserve the relation x = y. The expression then simplifies exactly to zero. This example shows why refining each wire's type is not sufficient in general. Sometimes the language of relationships between wires must be enriched.

One possible choice is a domain of linear constraints. Another is preserving symbolic equalities. Each permits certain operations and imposes certain costs. There is no reason to use the same abstraction for a numerical computation, an identity relation, and a coreference ambiguity.

Compositional Exactness Is a Stronger Condition

For certain operations, we can demand more than soundness:

α(F(s)) = F#(α(s)).

In this case, the summary after the operation can be computed exactly from the previous summary. For parity and addition, the property holds. For parity and the test “greater than a hundred,” it does not hold if we want an exact answer, because numbers with the same parity may give different answers.

This condition does not say that α preserves the entire state. It says that the discarded information is no longer needed by the operations and observations under consideration. Under these conditions, we can use the summary as the state of a compressed search. We will develop this idea in the chapter on sufficient interfaces.

Abstract interpretation thus offers two distinct contributions to the project: sound approximations for eliminating impossible branches and, in special classes, representations allowing exact continuation without the full state. The first contribution is more general. The second can produce spectacular reductions, but requires stronger proofs.

The Research Question

SOP Lang could associate one or more abstract interpretations with each family of operations. The same transfer operation could simultaneously update a concrete event table, an abstract description of possession, and a summary of source dependencies. The expander would use inexpensive summaries for guidance and richer representations when a necessary distinction arises.

This is a proposal, not a property obtained automatically by writing circuits. We must verify that the interpretations are compatible, that their results are not confused, and that moving to a richer representation does not lose the justification trail. A system with many uncoordinated abstractions may be harder to understand than one with a single simple representation.

8. Correlations: Information That Lives on No Single Wire

The Same Local Values, Different Worlds

Consider two situations involving two binary variables. In the first, the possible pairs are (0, 0) and (1, 1). In the second, they are (0, 1) and (1, 0). Considered separately, each variable can be 0 or 1 in both situations. Considered together, the situations are opposites: in the first the variables are equal; in the second they differ.

Any representation preserving only the individual sets identifies these two situations. If the final question concerns equality, that identification is wrong. This small example contains the essence of the problem of distributed representations: relevant information may reside in the relation, not in the components considered separately.

It does not follow that we necessarily need neurons or large vectors. The relation “equal” is a very compact symbolic representation of the dependency. Nor does it follow that symbols handle every dependency inexpensively. For certain families of relations, a compact form may be difficult to find or difficult to use.

A Conflict Invisible to Individual Descriptions

Let x, y, and z be binary values. We impose that x and y are equal, y and z are equal, but x and z differ. Each variable, viewed in isolation, continues to admit 0 and 1. Each relation considered separately admits combinations. Together, the relations are incompatible.

We can write binary equality as $x \oplus y = 0$, where $\oplus$ means addition modulo two: $0 \oplus 0$ and $1 \oplus 1$ are zero, while different values give one. Combining the three equations, each variable appears twice and cancels out. This leaves $0 = 1$. It is a certificate of incompatibility.

We did not explore all eight assignments to obtain the explanation. We used an algebra suited to the relation. In larger systems of such equations, linear elimination provides a general solving mechanism. The case does not prove the superiority of every global approach; it shows the inadequacy of a strictly individual summary.

The Frontier of a Circuit

Imagine cutting a partial circuit where future operations will connect. The values visible at this cut form the frontier. To decide which continuations are possible, we may need their values, the relationships among them, and the conditions under which they were produced.

A frontier is not necessarily a small list. If the problem requires comparing every person with every other, the relationships may grow rapidly. Nevertheless, certain structures allow compact summaries: an order relation, a partition into equality classes, a system of equations, or a graph with small separators. Searching for a sufficient interface becomes a search for the appropriate form of this frontier.

A separator is a set of variables through which two regions of the problem interact. If we know enough about the separator, we can treat the regions more independently. Intuitively, we need not know every detail of two cities to calculate their interaction if all relevant

exchanges pass through a few well-described ports. The important condition is “all relevant exchanges.” A hidden connection invalidates the separation.

Why “Holographic” Is Not Yet an Answer

A distributed representation can encode many relations in a single vector. This changes how information is accessed; it does not eliminate the need to preserve it. If a retrieval operation is approximate, its confusions must be measured. If relations are superposed, the resulting interference must be understood.

Likewise, a compact symbol can represent a very large relation only because its interpreter knows how to reconstruct or use it. Complexity must not be counted solely in the symbol's length. The interpreter's knowledge must be included. A one-byte identifier can refer to an enormous library; that does not mean the entire library has been compressed into eight bits.

The lesson for research is clear: compare available information, access costs, and types of error, not merely a value's apparent size. The question “symbolic or distributed?” becomes more fruitful when replaced with “which relationships are represented, which operations can use them, and with what guarantees?”

PART III

How We Compress Circuit Construction

From a tree of attempts to a shared graph: the conditions under which two histories can have the same relevant future.

9. Execution, Verification, and Discovery Are Different Problems

A Circuit with Perfect Wires May Still Be Hard to Construct

Let us impose radical discipline on outputs: every wire carries a single bit, every operation is a well-defined logic gate, and every connection is explicit. We can evaluate such a circuit for a given input by following its operations. But the question “is there an input that produces one?” may require a very different effort.

This is an important warning for SOP Lang. Combinatorial explosion does not arise only because values are vague texts or very rich objects. It can arise even with binary values and elementary rules. A disciplined format removes certain sources of confusion, but does not guarantee efficient search in every class of problems.

If an algorithm could find a satisfying input for any Boolean circuit in polynomial time, it would also solve standard satisfiability formulations encoded in this way. The book does not infer that the project should be abandoned. We infer only that a universal promise has far stronger theoretical consequences than an ordinary engineering optimization.

The Lesson of Cryptography

In cryptography, a transformation may be easy to execute and difficult to invert under the conditions assumed by the security model. Knowing the algorithm does not, by itself, provide an efficient method for

finding a preimage. One-way functions formalize this asymmetry as a hardness assumption, not as a proven truth about every apparently complicated function. [7]

For machines of understanding, the relevant analogy is limited and precise: transparent rules do not automatically eliminate the difficulty of discovering the right input, plan, or circuit. We are not claiming that language interpretation is a cryptographic function. We are using an established example in which the direction of computation matters fundamentally.

There is also the positive lesson of certificates. A result can be accompanied by an object easier to verify than to discover. For a SOP Lang circuit, the certificate may specify the premises used, operations applied, and conditions satisfied. But the certificate validates a formal relationship relative to those premises; it does not prove that an external source is telling the truth.

Four Places Where We Can Hide the Cost

We can hide cost in the library: each operation appears simple because someone previously built a very powerful mechanism. We can hide it in the representation: a compact state refers to an enormous object. We can hide it in preprocessing: queries are fast only after costly compilation. We can hide it in the evaluator: verifying a contract is itself a difficult problem.

An experimental result must declare where these costs lie. For a system used for a million questions, expensive compilation may be excellent. For a single new question, the same solution may be uneconomical. There is no contradiction; the usage regime changes.

Likewise, an algorithm using a table of all possible answers may have fast queries. Without further arguments, that does not make it an efficient algorithm for learning or constructing the table. The ideal sufficient interface may exist mathematically and be too difficult to obtain in practice.

What Kind of Efficiency We Seek

The proposed program seeks efficiency conditional on structure. There may be domains with few relevant state classes, relations with simple algebra, small frontiers, or reusable libraries. In such domains, search compression can be decisive. In other domains, the system must recognize budget limits and provide an explicitly incomplete result.

A reasonable objective is not "we will never branch," but "we will not branch separately over histories we can prove have the same relevant future." This reformulation preserves the ambition while removing an unjustified claim to make every computation easy.

There is another benefit not reducible to speed: even when search remains expensive, a disciplined representation can reveal why it is expensive. We can identify the relation connecting many variables, the unresolved ambiguity, or the operation for which we lack a sufficient summary. Explaining the cost is itself a research result.

10. Continuation Equivalence: When Two Histories Are the Same State

The Example of an Automaton Reading Digits

Imagine a device that reads a string of digits and must decide at the end whether the represented number is divisible by three. It need not retain the entire string. It can retain the remainder on division by three. Very different histories thus arrive at the same internal state.

Why is this legitimate? If two prefixes give the same remainder, any suffix of digits appended to both again produces the same final remainder. No permitted future can distinguish the prefixes for the question under consideration. They do not have the same text, length, or numerical value. They have the same relevance for continuation.

This idea is central to automata theory and the family of Myhill–Nerode theorems. Extensions to register automata and symbolic traces show that equivalences may require preserving relations, not merely a simple finite label. [8]

Contextual Equivalence

For two partial circuits A and B, we can ask whether every permitted continuation K produces the same relevant observation when attached to A or B. Schematically:

$A \equiv_Q B$ when, for every permitted K, $Obs(K[A]) = Obs(K[B])$.

Q indicates the family of questions or observations that matter. If we observe only the numerical result, two circuits may be equivalent. If we also observe energy consumption, latency, external effects, or provenance, that equivalence may disappear. There is no equivalence without specifying what we are willing to ignore.

Take two routes arriving at the same station at the same time. For continuing a journey, they may be equivalent. For reimbursing expenses, they may differ. For a question about places visited, history is essential. The same summary does not serve every purpose.

Why This Is More Than Memoizing Results

Ordinary memoization reuses a function's result for the same input. Contextual equivalence can merge different inputs and different histories when their difference does not affect continuation. It is a semantic compression of state, not merely avoidance of identical repetition.

For the sum of a sequence, we can retain the partial sum. For its mean, the sum alone is insufficient: we also need the number of elements. For the median, the sum and count are not generally sufficient. These examples show that the size of the required state depends on the future question and the permitted operations.

A chosen summary can be tested by searching for a pair of histories it merges and a continuation that separates them. Such a triple—two histories and a continuation—is a concrete counterexample to sufficiency. We need not discuss vaguely whether the summary “seems to preserve meaning.” We can seek the exact situation in which it does not.

Similarity Is Not Equivalence

Two expressions may give similar results on observed examples and different results on others. The functions $x + 1$ and $3x + 1$ coincide at $x = 0$, but not at $x = 1$. A semantic identifier constructed solely from the first example would merge them unjustifiably.

A feature vector or fingerprint computed over a set of tests can be useful for finding candidate equivalences. Exact verification can follow. If the domain is finite and tested exhaustively, observations can decide equivalence on that domain. If the domain is unbounded, a finite number of tests does not automatically become a universal proof.

Likewise, a source hash detects syntactic identity with the known technical risks of the scheme used; it does not prove that different sources have the same meaning. An auditable machine must distinguish representational equality, tested equality, and proven equality.

Bisimulation and Interactive Systems

For a process interacting with its environment, we care about more than a final result. Two states may be considered equivalent if every observable step from one can be matched by a step from the other, preserving the relation between the resulting states. This is the intuition of bisimulation.

The exact conditions depend on the model: agent-chosen actions, environmental actions, nondeterminism, and probabilities are not interchangeable. For SOP Lang, the practical lesson is that an interface suitable for a pure function may be insufficient for an agent capable of acting. Relevant possibilities for interaction must also be preserved.

The definition through all continuations is a semantic ideal. It tells us what should be preserved. It does not yet give us an efficient procedure for deciding equivalence. The next chapter seeks sufficient local conditions that can be checked on the library's operations.

11. Sufficient Interfaces: A Small Proposition with Large Consequences

A Summary's Contract

Let s be the complete state of a partial construction and α(s) its summary. For every permitted operation F, assume there is an operation on summaries, F#, such that summarizing after execution coincides with executing on the summary. Also assume that the final answer can be computed from the summary through a function h.

α(F(s)) = F#(α(s)); R(s) = h(α(s)).

The first condition says that the summary can be continued. The second says that it preserves the final observation. Together, they allow the complete state to be replaced with the summary for any sequence of operations from that library.

The argument is a simple induction. Initially, the summary is correct by definition. If the summary corresponds to the state before a step, the first condition preserves that correspondence after the step. Repeating this maintains the correspondence through to the end. The second condition then produces the correct answer. There is no mysterious appeal to intelligence here, but a property of the representation and operations.

The Example of Affine Functions

A function of the form f(x) = ax + b can be represented by the pair (a, b). If permitted operations add a constant to the result or multiply it by a constant and then add another, the function remains affine. We do not need the list of all previous operations to continue the functional computation.

For example, if f(x) = 2x + 3 and we add 5, we obtain 2x + 8. If we then multiply by 3 and add 1, we obtain 6x + 25. The summary updates through arithmetic on two coefficients. Different histories producing the same coefficients have the same behavior for permitted continuations.

But if we add the operation “square the result,” the affine family is no longer closed. The pair can no longer describe every result exactly. We can enrich the representation to polynomials or prohibit the operation at this level of the search. This is an explicit decision, not a hidden exception.

When the Tree Becomes a Graph

Assume a search with n stages. At each stage there are at most M distinct summaries, and at most r extensions are attempted from each. If we merge all constructions with the same summary at the same stage, the number of extension evaluations is at most on the order of nMr.

Number of extensions evaluated $\leq n \cdot M \cdot r$.

This is not a theorem about all program synthesis. It assumes that the summarized state contains everything affecting future steps, including the stage or budget when relevant. It assumes a precisely specified library and a layered search scheme. The cost of evaluating, normalizing, and comparing summaries must be multiplied in separately. For an arbitrary graph, we cannot replace its entire structure with a stage number without justification.

If M grows exponentially with problem size, the bound does not provide polynomial efficiency. If the summary must be discovered through a search harder than the problem itself, the online advantage may hide a prior cost. If we count every path, the resulting numbers may require more and more bits, and operations on them no longer have constant cost.

Why the Frontier Must Be Summarized

A construction may produce two values, x and y. A future operation may compare them. Then a separate summary of x and a separate summary of y are insufficient if their relationship has been lost. State $\alpha(s)$ must represent their shared frontier in the context of permitted continuations.

Anything affecting whether a continuation is permitted must also be included. If an operation consumes a resource, two circuits with the same output but different remaining resources are not equivalent for

all future plans. If a conclusion must be supported by a particular class of sources, relevant provenance cannot be discarded merely because the computed value coincides.

We can nevertheless separate levels. Value equivalence may suffice to guide a search. To emit the result, we retain a witness and check provenance. But if future choices themselves depend on provenance, it must be included in the state used by the search. Otherwise, compression may eliminate the only acceptable variant.

The Research Program Hidden in the Proposition

The underlying mathematics is not new: it is related to dynamic programming, homomorphisms, and semantic congruences. SOP Lang's possible contribution would be identifying such interfaces for useful families of reasoning and learning them from texts, examples, and counterexamples.

We should investigate whether a system can begin with a rich representation, observe regularities among continuations, and propose a smaller summary. A verifier would then try to find a counterexample. An accepted summary could become the interface of a reusable circuit. Through repetition, the system would learn not just programs, but forms of state in which programs become easier to construct.

This is the book's most ambitious hypothesis. It is not confirmed by proving a property for a human-provided representation. The experiments must be organized around precisely the gap between “a sufficient summary exists” and “the system discovers it at a useful cost.”

12. Dynamic Programming, Knowledge Compilation, and Graphs of Alternatives

The Same Subproblem, Several Paths to It

Dynamic programming avoids repeatedly solving equivalent subproblems. To find an inexpensive path, for example, we do not necessarily retain every route reaching a node. Under suitable conditions, we retain the relevant minimum cost and a predecessor. But those conditions matter: if the objective requires never revisiting a city, the visit history may become part of the state.

The same lesson applies to circuits. We cannot merge constructions merely because they produce the same value now. We must know that the discarded past no longer affects the permissibility or value of the future. Dynamic programming succeeds when a well-chosen state makes subproblems independent of irrelevant historical details.

Knowledge Compilation

We can transform a knowledge base into a representation in which certain questions are easier to answer. Compilation may be expensive, but repeated queries may justify the investment. Darwiche and Marquis's map compares precisely the trade-offs among representational compactness, efficient queries, and available transformations. [10]

There is no universally winning format. A structure may allow rapid compatibility checks while making updates difficult. Another may be compact for one family of formulas and very large for another. The choice must start from anticipated questions and changes.

For SOP Lang, a book or case file could be compiled into a graph of entities, events, and justifications. But ingestion costs, interpretation corrections, and adaptation to unanticipated questions must be included in the evaluation. A perfect summary for today's questions may be a poor archive for tomorrow's research.

The Width of Relations

In many problems, difficulty depends on how many variables must be handled together during an elimination or separation step. The notion of treewidth formalizes one aspect of this width. AND/OR decompositions and AOMDDs exploit conditional independences for more compact representations and computations. [11]

Intuitively, if two large regions interact through only two variables, we can summarize each region relative to those two variables. If the interaction passes through almost every variable, the summary may become enormous. The total number of variables is not the only indicator of difficulty; the shape of their connections matters.

Nevertheless, graph width does not exhaust exploitable structure. Relations may have algebraic properties or functional dependencies that reduce computation even when the graph appears dense. Work on

variable elimination in the presence of functional dependencies illustrates this possibility. [12] The system must therefore be able to recognize both structural separation and the appropriate algebra.

E-Graphs: Many Equal Expressions, One Shared Space

An e-graph retains families of expressions that the accepted rules declare equivalent. Instead of choosing immediately between a + 0 and a, we can represent them in the same class and continue exploring. Equality propagates through compatible contexts. We later extract an expression suited to a cost criterion. The egg system is a practical reference point for this organization. [17]

This differs from retaining alternative interpretations. Expressions in the same equality class must be equal under the semantics and assumptions in use. Two interpretations of a pronoun are not automatically equal; they are alternatives that may produce different answers. Both can share a graph, but sharing does not authorize confusing their meanings.

Rewrite rules must also be checked. Transforming a division x/x into 1 requires a condition such as x being nonzero under appropriate numerical semantics. With effects or special values, even familiar rewrites may require care. A library of unverified equalities turns compression into a source of systematic errors.

Compression Does Not Eliminate Every Explosion

A shared graph can become very large. Equality saturation sometimes needs guidance to reach complex optimizations within practical budgets; research on sketch-guided methods explicitly studies this issue. [30] Saying “we use an e-graph” is not a complete answer to the problem of controlling search.

SOP Lang could combine representations: equality classes for provably identical expressions, packed graphs for alternative interpretations, and abstractions for filtering. What matters is that every connection has a declared meaning. Equality, possibility, implication, and similarity must not become four labels for the same ambiguous edge.

13. Types, Backward Requirements, and Counterexample-Guided Refinement

Types as Initial Filters

A type can state that an operation takes a person and an event, not two numbers. A richer type can state that the event is a transfer and the person its recipient. Refinement types add logical conditions to basic properties. In synthesis, they can eliminate incompatible compositions before concrete execution. Research on synthesis from refinement types explores this direction. [13]

Yet type compatibility does not mean an operation is useful for the goal. Thousands of operations may take and produce texts. A discipline that sees only the type “text” does not guide search sufficiently. We need contracts stating the relationship between input and output.

Propagating Requirements Backward

If the objective requires a positive result and the proposed operation computes $x - 5$, a necessary requirement is $x > 5$. If the objective requires identifying the possessor of an object after a transfer, the operation needs the identities of the object and recipient and the relevant temporal conditions. The resulting requirements can become subproblems.

For a deterministic operation, the weakest precondition of a goal is the set of inputs guaranteeing that goal after execution. We need not be able to compute it exactly in every case. A necessary condition may suffice to reject certain branches. A sufficient condition may allow others to be accepted soundly. These two directions of approximation must be labeled distinctly.

An expander can bring together what is possible going forward and what is necessary going backward. If the two do not intersect, the branch is impossible within the model. If they intersect, the branch remains a candidate. A nonempty intersection does not prove the existence of a concrete execution satisfying all relations lost through approximation.

CEGAR: When an Alarm Reveals the Model's Limits

Suppose interval analysis sees x and y in [0, 100] and reports that x − y could be negative. The concrete model, however, contains y = x. The abstract counterexample, such as x = 0 and y = 100, is infeasible. Instead of declaring the program wrong, we enrich the abstraction with the missing relation.

This is the intuition of CEGAR: begin with an abstract model, check a candidate counterexample, and refine the model when the counterexample is spurious. The foundational paper formulates this cycle for system verification. [16] In SOP Lang, a related mechanism could introduce a semantic distinction precisely when its absence produces a suspicious derivation.

Not every refinement should be accepted. One that adds almost the entire concrete state may solve the current case and destroy the advantage. We must measure how many errors it eliminates and how much it enlarges the representation. Refinement is an investment, not a cost-free good.

CEGIS: When a Candidate Program Is Wrong

In counterexample-guided synthesis, we construct a candidate satisfying the current examples. A verifier seeks an input on which the candidate violates the specification. If it finds one, we add the case and construct another candidate. Here we repair the candidate program, not necessarily the analysis abstraction. Syntax-guided synthesis explicitly separates the grammar of permitted programs from the behavioral specification. [15]

The distinction between CEGAR and CEGIS is practically useful. If a false interpretation is admitted because the summary lost a relation, the problem lies in the abstraction. If the summary is adequate but the chosen circuit computes something else, the problem lies in the candidate. If the text was misinterpreted at the outset, both mechanisms may flawlessly verify the wrong problem.

A Cycle for Learning Interfaces

We can propose a third object of refinement: the very relation by which we merge states. The system proposes that two frontiers are equivalent. A verifier seeks a continuation distinguishing them. When

it finds one, we add the missing feature to the interface. When it finds none within a limited budget, the result must be labeled “no counterexample found,” not automatically “equivalence proved.”

TYGAR is a precedent for using type-guided abstraction refinement to control synthesis. [14] The proposed research extends the question to semantic representations of situations and circuit frontiers, without assuming that this transfer is automatic.

A success would be repeatedly discovering small interfaces that generalize to new problems. An informative failure would be a situation in which every counterexample requires a new exception and the representation grows almost in proportion to all memorized cases. This would show that we have not yet found the right structure, or that the problem family does not offer it in the form sought.

PART IV

Language, Memory, and Distributed Representations

What must be preserved from text, uncertainty, and memory so that new questions remain possible.

14. From Sentences to Situation Circuits

Text Is Not Yet the Knowledge Base

A sentence is a source of information, not an already validated table of facts. “Maria believes Ana has left” contains a report about a belief. It does not automatically authorize inserting the fact “Ana has left” into the accepted description of the world. “Ana would have left if the train had arrived” requires another structure. A system extracting only names and verbs may lose precisely the essential difference.

In SOP Lang, ingestion should construct circuits that propose and verify interpretations, whose execution produces materialized representations. We can have tables for entities, events, roles, temporal relations, and sources. These tables are values of circuits. They must not be confused with the source of the circuit producing them.

A transfer episode may have an event identifier, an agent, a recipient, an object, a time interval, and a textual source. References to people and objects function conceptually like links between database tables. This allows two sentences to refer to the same object without repeating its entire description.

A Complete but Small Example

Consider the text: “Ana had the key on Monday morning. At noon she gave it to Maria. In the evening, Maria entered the library.” An initial representation would identify two people, a key, a library, a possession state, and two events. The pronouns in the second sentence must be linked to the key and the relevant people.

For “who had the key after noon?” the circuit would use the transfer rule and temporal order. For “who owned the key?” the text may be insufficient: physical possession does not necessarily imply ownership. For “did Maria use the key to enter?” entering the library does not by itself establish the mechanism by which the door was opened.

These differences should appear in the structure of the representation, not merely in a verbal warning added at the end. A fact about possession, an assumption about an instrument, and an unanswered question have distinct statuses. Fluent wording must not flatten them.

Rules as Reusable Circuits

A transfer rule can receive a validated event and a prior state, then produce a subsequent possession state. The rule may require conditions: the object exists, the participants are identified, the event is actual rather than merely hypothetical, and the relevant time is sufficiently established. These requirements make the rule selectable and verifiable.

Such a rule must not be treated as a universal law of language or the world. “Ana gave Maria an idea” does not transfer possession in the same sense as a key. “She gave her the key but immediately took it back” requires a subsequent event. An adequate contract defines the domain in which the rule is used and allows abstention outside it.

In a learning system, some circuits may initially be generated by a language model or a researcher. This fact must be declared. Producing an auditable circuit does not mean that the system learned the rule autonomously. Research must distinguish generating, validating, and reusing rules.

Datalog as an Intuition of Derivable Knowledge

Datalog offers a simple model: we begin with base relations and rules producing new relations. If A is B's parent and B is C's parent, we can derive that A is C's grandparent. We repeat rule application until no new facts appear. The official Soufflé tutorial illustrates this relational organization. [20]

For positive rules over a finite universe, the computation can be understood as reaching a fixed point. We need not choose a single proof path to know that a fact is derivable. We can preserve several justifications and reuse shared subderivations.

However, unbounded arithmetic, generating new terms, negation, and external effects change termination conditions and semantics. Datalog is not synonymous with all of natural language. For SOP Lang, it can inspire or implement a class of derivation operations without becoming a universal explanation of interpretation.

Uniformity Without a Hidden Second Language

SOP Lang's ambition is for interpretations, rules, and strategies to be expressible through composable circuits. This does not require eliminating every primitive. A system needs basic operations whose meanings are established outside their own expansion. What matters is that this foundation is explicit and small enough to analyze.

If most actual reasoning resides in an opaque interpreter and the circuit merely passes it text, uniformity becomes cosmetic. If, instead, circuits expose entities, conditions, alternatives, and justifications, they become objects of research. A useful test is to ask what a researcher can verify about a result without taking the component that proposed it at its word.

15. Ambiguity Without Multiplying All the Worlds

Why Choosing Immediately May Be a Mistake

Without additional context, the text “Ana gives Maria the key. She leaves for the station” admits at least two interpretations of the pronoun. For “who has the key after the transfer?” both may produce the same answer under the explicit model in which transfer changes possession and departure does not. For “who leaves?” the difference is essential.

A system that immediately resolves the pronoun may introduce an unnecessary error. A system that fully develops two separate worlds may duplicate computations independent of that choice. The concep-

tual solution is to retain a local alternative and share the common structure. We resolve the ambiguity when the objective requires it or relevant information appears.

This does not mean every ambiguity can be ignored until the end. Some decisions affect event type, object identity, and rule applicability. Then conditions connecting consequences to each alternative must be maintained. Sharing is correct only if these dependencies remain recoverable.

The Intuition of Chart Parsing

In syntactic analysis, the same text span may participate in several interpretations. A chart retains partial results for text intervals so that shared substructures are not reconstructed at every use. A packed graph can describe many parse trees without enumerating them separately.

For fixed context-free grammars, classical algorithms may have a number of cells quadratic in text length and a number of combinations cubic in text length, with grammar-dependent factors. This property does not automatically transfer to unbounded semantics, coreference, or world knowledge. Nevertheless, the sharing mechanism remains instructive. Goodman's semiring treatment shows how the same parsing structure can support different computations. [18]

For SOP Lang, spans may produce candidate circuits rather than just grammatical labels. Two expressions describing the same event may reuse fragments. But if their equivalence is merely assumed by a model, it must not be confused with verified equality.

What a Semiring Is and Why It Matters

A semiring provides two operations: one for alternatives and one for combining the components of a derivation. When considering the existence of a parse, alternatives combine through “or,” and required components through “and.” To count derivations, we add alternatives and multiply the numbers of independent combinations. For minimum cost, we take the minimum over alternatives and add component costs.

The same graph can thus answer different questions: does a derivation exist, how many derivations are there, which is cheapest? But each computation has precise semantics. Two derivations of the same conclusion are not necessarily two distinct worlds. A count of derivations is not automatically a probability.

If a shared component appears in two branches, we cannot treat its two occurrences as independent evidence without analysis. For probabilities, dependencies and overlapping events must be modeled. A sum of scores may be a useful heuristic, but must not be labeled a calibrated probability without an appropriate model.

Provenance as an Expression of Dependencies

We can denote sources by symbols p, q, and r. If a conclusion requires p and q, its provenance may contain the product pq. If it follows either from p and q or from r, we can retain the expression pq + r. Here the signs describe two ways of constructing a justification, not ordinary arithmetic of truth.

Semiring provenance semantics develops this idea for logics and databases. Extensions to negation and fixed points show that algebraic properties must be chosen carefully, not mechanically extrapolated from the simple positive case. [19, 32]

For a researcher, the benefit is concrete: when source p is withdrawn, we can see whether r still supports the result. When two sources appear different but originate in the same document, we can avoid interpreting them as independent support. When an answer has a circular justification, the derivation structure can make that circle visible.

What We Compress and What We Preserve

A packed graph need not retain every detail of every interpretation in every node. It can use shared references, branch conditions, and reconstruction operations. But there must be a way to verify that an answer comes from a coherent interpretation, rather than an arbitrary combination of fragments from several incompatible interpretations.

The research hypothesis is that many natural ambiguities are irrelevant to many questions, and that the system can exploit this structure. We do not assume all are local or inexpensive. A good experiment will

increase the number of ambiguities and the number of relations connecting them separately. Ten independent ambiguities and ten globally correlated ambiguities are different problems.

16. Time, Negation, Quantifiers, and Causality

Time Is Not a Decorative Attribute

“Maria has the key” and “Maria had the key yesterday” cannot be inserted as the same fact. A state holds over an interval or within a temporal context. An event can change the state. A report may be made long after the event. These three times—validity, occurrence, and reporting—must be distinguished when they affect the answer.

A simple model can use intervals and relations such as before, after, simultaneous with, or contained within. It need not always know an exact date. From “the transfer occurred before the departure” and “the departure before closing,” it can derive the order without inventing times.

Persistence is a model rule: a state continues until a relevant event changes it. It is not an automatic consequence of every text. If we do not know what happened between two times, a persistence assumption must be marked and possibly revised. Persistence rules differ across domains.

Negation Is Not the Absence of a Record

If the database does not contain the fact that Ana is in the library, it does not generally follow that Ana is not there. Absence of evidence and evidence of absence are different things. In an administrative database complete for a specific purpose, we may adopt a closed-world convention. When reading fragmentary texts, this convention is often inappropriate.

The system should distinguish “supported,” “refuted,” “undecided,” and “incompatibility between sources,” if these are the application's epistemic states. They need not all be represented through a particular four-valued logic. They must not be confused in the answer.

Negation also changes monotonicity. If a rule says “assume available unless marked reserved,” subsequently adding a reservation retracts the conclusion. The circuit must preserve its dependence on this controlled absence. It cannot treat the result as a permanently derived positive fact.

Quantifiers Change What Must Be Preserved

“Every researcher read a book” may mean that each read a possibly different book. It does not imply that the same book was read by everyone. The order of “for every” and “there exists” changes the meaning.

A representation extracting only the relation reads(researcher, book) may lose the domain and the dependency of the choice. For some questions, this information is sufficient: every researcher read something. For others, such as organizing a seminar on a shared book, it is not. This is another case of an interface sufficient for one question and insufficient for another.

Comparatives and conditionals add similar difficulties. “Faster than” requires a comparison target and a criterion. “If it rains, the meeting moves” asserts neither that it is raining nor that the meeting has moved. Circuits must represent conditional relationships without prematurely materializing their consequences.

Reporting and Points of View

“Ana says Maria has left” produces at least a fact about the act of reporting. It may also produce a hypothesis about the departure, with the appropriate source and status. If Maria says the opposite, the conflict may be between reports about the world without either act of reporting being falsely described.

A representation using contexts or indexed worlds allows this structure to be preserved. Not every conversation needs to generate a complete philosophical universe. It is enough for statements to carry the context that changes their use: accepted by the system, reported by a source, assumed in a scenario, or explicitly negated.

Computational Dependency Is Not Automatically Causality

In a circuit, an output depends on inputs by construction. In a model of the world, a correlation between two variables does not by itself say what happens if we intervene on one. A system can predict an umbrella from rain and rain from an umbrella, but opening the umbrella does not produce rain in the same sense.

Structural causal models study mechanisms and interventions, not just observational conditioning. Pearl explains the distinction between observing a value and changing the mechanism that produces it. [27] For SOP Lang, a causal edge must come from a model or a declared causal hypothesis, not merely from the order of text-processing operations.

A useful conceptual experiment would construct two worlds with the same observational regularities in the available data but different responses to intervention. If the system confuses them, the predictive representation is insufficient for causal questions. That does not make it useless; it defines its scope.

17. Symbolic, Neural, Distributed: Three Axes, Not Two Camps

A Vector Can Be the Value on a Wire

A neural network can be described as a graph of operations on tensors. A tensor is an organized collection of numbers. A conceptual wire may carry the entire tensor, or we may describe its coordinates separately. Changing granularity does not eliminate the distributed character of the representation. Transformer operations are an established example of computation expressible through such dependencies. [21]

Consequently, a value having an SSA name does not mean its meaning is localized in a single symbol. An identifier can name a vector across which information is distributed. Conversely, a symbolic relation can depend on multiple objects and context. The location of the name is not the location of meaning.

At least three axes must be distinguished: discrete or continuous representations; explicit or implicitly learned meanings; exact or approximate operations. A system can be symbolic and probabilistic, vector-based and deterministic, or hybrid. A binary classification loses these combinations.

Why Distributed Representations Can Be Useful

A distributed representation can give similar objects nearby descriptions and allow retrieval from incomplete cues. In a circuit-selection system, this can be very useful: the problem need not match the operation's description word for word for a relevant candidate to be found.

VSA/HDC explores operations on high-dimensional representations. A binding operation can associate a role with content, while a bundling operation can combine several elements into a representation. Research on generalized holographic reduced representations also investigates properties of binding and compositionality. [22]

For intuition, imagine a representation combining “agent: Ana,” “action: gives,” and “object: key.” We want to ask who the agent is or which object appears. If retrieval is approximate, we must measure when Ana is confused with Maria, when roles are reversed, and how error grows with the number of superposed relations.

Compression Does Not Defeat the Arithmetic of Information

A memory with b bits has at most 2^b distinct states. If we want to distinguish more situations than that, we must use additional structure, increase memory, allow losses, or restrict the questions. A representation using real numbers with ideal precision is not a cost-free finite physical memory.

This argument does not reject distributed memory. It says only that its advantage must be described in terms of redundancy exploited, tasks preserved, and errors accepted. The same requirement applies to symbols. A symbolic model using a compact rule may gain enormously when the data obey that rule; when they do not, exceptions may occupy considerable space.

An honest comparison fixes the memory budget and numerical precision, then measures retrieval and generalization. It does not compare an exact symbolic structure preserving every distinction with an approximate vector without specifying the tolerated loss.

Gradients and Discrete Search

In a differentiable neural model, parameters can be adjusted using information about how error varies. This optimization route differs from enumerating discrete circuits. But optimization does not automatically guarantee reaching the global solution, and the existence of a useful gradient depends on the representation and objective.

SOP Lang could use learned models to propose operations, recognize patterns, and suggest abstractions. A separate verifier would check the relevant contracts. An incorrect neural proposal would consume budget, but should not automatically become an accepted fact.

There is an unavoidable limit: exact verification is not available for every natural-language claim. If the task is summarizing an ambiguous essay, we do not always have a complete specification. The system must then combine partial checks, sources, and human assessments, marking what it cannot certify.

An Experiment That Would Establish Something Useful

Instead of asking which paradigm is generally superior, we can fix a task: retrieving roles from composed events, answering new questions, and detecting a contradiction. We compare an exact relational representation, a vector representation, and a hybrid. We keep the input information identical and measure cost, error rate, and explanatory capacity.

A victory for vectors in approximate retrieval would justify using them there. A victory for exact relations in contradiction detection would justify a symbolic core for that role. The goal is an argued allocation of responsibilities, not defending allegiance to a school.

18. Sufficient Representations for Prediction and Future Questions

Sufficient Statistics Through a Familiar Example

If we want the mean of a list, the sum and number of elements suffice. We do not need their order. If we want to know whether the list was increasing, those same two numbers are insufficient. Sufficiency is always relative to something we want to compute or infer.

In statistics, a sufficient statistic preserves relevant information about a parameter within a specified model. For our research, the analogy is useful but must not be extended without qualification: a circuit interface may need to preserve behavior for every permitted continuation, not merely information about a parameter within a fixed distribution.

There are thus three related but distinct notions: exact sufficiency for a computation, sound approximation of possibilities, and preservation of predictive information within a probabilistic model. They can cooperate, but cannot replace one another.

Predictive States

Two histories of a process may be considered equivalent if they induce the same distribution over future observations under the model's conditions. Computational mechanics studies such predictive states and their relationship to representing process structure. [9]

The intuition resembles continuation equivalence: different histories are merged when they no longer change relevant predictions. But equality of distributions is not the same as equality of values observed on a few examples. Estimating it from finite data introduces uncertainty and depends on assumptions about the process.

The term “causal states” used in this literature must not automatically be confused with identifying intervention effects in structural causal models. A state sufficient for predicting observations may be insufficient for choosing an intervention. The distinction from the previous chapter remains necessary.

Information Bottleneck

The information bottleneck method seeks a representation Z of data X that preserves useful information about a target Y while discarding details from X. Intuitively, we want a small but still predictive summary. A classical objective balances information between X and Z against information between Z and Y. [24]

Conceptual trade-off: minimize $I(X; Z) - \beta I(Z; Y)$.

Here I describes mutual information, meaning informational dependence between variables in the modeled distribution. β determines how much we value predicting the target relative to compression. The formula does not guarantee that every important meaning is preserved. What does not appear in the target or evaluation distribution may be discarded.

The deterministic information bottleneck variant explores an objective based on representation entropy, favoring another form of compression. [25] “Deterministic” refers here to the representation and the method's formulation, not to turning the world into a process without uncertainty or automatically proving semantic equivalence.

Changing the Question Is the Decisive Test

A document summary may preserve the theme and general conclusion while losing a rare exception. For a reader interested in the idea, this compression may be good. For a researcher asking precisely about the exception, it is insufficient. A single average summarization score can conceal this difference.

For SOP Lang, the proposed solution is to separate persistent memory from the question-adapted view. The source and richer representation remain accessible. For the current question, we construct a smaller interface. When the question changes, we can refine or reconstruct the view rather than pretend the old summary is universal.

This organization costs memory and administration. It must be compared with alternatives: retaining all derivations in full, retaining only the text, or irreversible compression. The criterion is not an abstract preference for rich memory, but the relationship between cost and the number of future questions the system can answer correctly.

A Hypothesis About Understanding

We can now formulate an exploratory hypothesis: part of understanding consists in discovering representations that preserve what remains invariant under relevant variations and set aside details that do not change the consequences under consideration. This explains why an expert can see the same structure in problems with different appearances.

The hypothesis does not say that all details are noise or that every problem has a small essence. Sometimes an exception is the very essence of the question. A mature system should also learn when compression ceases to be legitimate. The ability to reopen a distinction can be as important as the ability to eliminate it.

PART V

A Conceptual Architecture for SOP Lang

A proposed organization of knowledge, synthesis, and verification; controlled observations and explicit limits.

19. What a Semantic Interface Must Carry

More Than a Value, Less Than the Entire History

A semantic wire should let a continuation discover what it can use without inspecting the entire past again. Sometimes this means an exact value. At other times it means a relation, a set of alternatives, or a conditional result. The proposal is not that every wire must carry the same enormous structure. It is that there should be a common discipline governing the questions the interface must be able to clarify.

The first question concerns the nature of the content: is it a person, an event, a relation, a candidate interpretation, or a circuit? The second concerns the guarantee: exact content within the model, an approximation including all possibilities, a hypothesis checked only on examples, or a heuristic proposal? The third concerns dependencies: under which premises and in what context does it hold?

Some information can be accessed through references rather than copied onto every wire. Provenance can reside in a shared graph. The schema can be declared by the type. A relation can be stored once and used by several values. Interface discipline concerns the information's availability and meaning, not an obligation to duplicate it physically.

Three Interface Levels

The operational level describes what a component can consume and produce. The semantic level specifies the relationship between input and output. The epistemic level states why we are entitled to use that relationship in the current case. A component may be perfectly executable at the first level and insufficiently justified at the third.

For example, an extractor receives text and produces a transfer event. The operational contract is clear. The semantic contract should state that the event corresponds to a particular fragment. Its epistemic status may remain “candidate interpretation.” A subsequent possession rule may be exact relative to the event, but the final answer inherits the interpretation's uncertainty.

This organization prevents a common error: turning an assumption into certainty by passing it through many exact operations. Ten correct steps following an uncertain premise do not make the premise certain. A circuit must preserve the dependency, not launder uncertainty through computation.

The Library and the Interpreters

A circuit library should expose which problems it can solve, under what conditions, and which representations it uses. The expander can consult these contracts to find operations compatible with the objective. A composite operation can be expanded into a subcircuit when detailed analysis is needed.

Interpreters carry out operations under chosen semantics. One interpreter may compute finite arithmetic exactly. Another may evaluate constraints. Another may propose a linguistic interpretation. Uniform invocation does not imply uniform guarantees. The system must be able to inspect the difference.

An important question concerns the trusted base: which operations and verifiers are assumed correct for the rest of the explanation to work? A small base can be tested and analyzed more effectively, but it does not disappear through metaprogramming. A circuit verifying another circuit still relies on its own operations' semantics. A uniform representation does not remove this foundation.

The Expander Need Not Immediately Choose a Single Program

The main proposal is for the expander to construct a shared graph of possible continuations. Some nodes represent unresolved obligations; others, intermediate results; others, classes of equivalent constructions. Links indicate compositions, alternatives, or justifications without confusing these meanings.

The objective generates requirements. Available information generates possibilities. Operation contracts allow the two to meet. Inexpensive abstract representations eliminate incompatible candidates. Exact representations or stronger verifiers control acceptance. When an interface is too impoverished, it is refined or the case is split into alternatives.

This process can be deterministic in selecting and checking steps, use learned heuristics, or combine them. Determinism is a property of the procedure, not a guarantee of low cost. A fully deterministic procedure may explore exponentially many cases.

What Progress Means

Not every useful step immediately shrinks the set of possibilities. Discovering an ambiguity may increase the number of represented interpretations. Adding a relation may enlarge the state while reducing subsequent search. Introducing a hypothesis may create new branches while enabling a previously missing explanation.

That is why a single indicator such as “total uncertainty decreases at every step” would be too rigid. We can separately measure resolved obligations, verified distinctions, soundly eliminated candidates, estimated remaining cost, and result quality. Reopening a mistaken decision can be progress even if the graph temporarily grows larger.

A machine of understanding must be able to say not only “I found a circuit,” but also “this circuit satisfies the objective under these assumptions; these are the alternatives that remain relevant; here is the limit of verification.” This is the form of result the architecture seeks.

20. Anatomy of a Question Answered Without Inventing What Is Missing

The Situation

We will follow a complete conceptual example. The text says: “On Monday morning, Ana had the laboratory key. At noon, she gave the key to Maria. She left for the library. On Tuesday morning, the guard found the key on the laboratory table.” The initial question is: “Who had the key immediately after Monday's transfer?”

For this example, we accept restricted semantics: “giving the key” describes a completed physical transfer; the key mentioned is the same one; “immediately after” excludes a subsequent unmentioned event at the same moment. These are the model's premises, not truths about every use of the words. The system must be able to display them or link them to domain rules.

We will not claim that a prototype has already completed all the stages below. The example describes behavior the research program seeks to achieve and test.

The Initial Interpretations

Ingestion identifies the people, object, places, and events. It constructs an interpretation of the transfer to Maria and two alternatives for the pronoun “she”: Ana or Maria. Tuesday's event is represented separately, after all Monday's events.

Instead of copying the entire situation into two independent worlds, the system retains the shared structure and a local choice for the agent of departure. Consequently, any conclusion depending on that choice carries the appropriate condition. A fact independent of it can be computed once.

The question specifies an object, a time, and a required relation: possession of the key immediately after the transfer. The expander searches for operations capable of producing that relation. The transfer rule is a candidate because its output has the appropriate form and its conditions are compatible with the available information.

Propagating Requirements

The rule requires an identified recipient, an object, and an actual rather than hypothetical event. These conditions are satisfied within the accepted model. It also requires locating the question immediately after the transfer. The temporal relationship is supplied by the question's wording itself.

The agent of departure for the library does not appear among the necessary requirements. For this rule library, the system can prove that both interpretations of the pronoun lead to the same possession im-

mediately after the transfer. It therefore need not choose who left. This is not arbitrarily ignoring an ambiguity, but proving its irrelevance to the current observation.

Tuesday's event does not retrospectively change the answer. It may provide information about a later state, but does not prove who returned the key. The circuit must not invent a complete trajectory merely to connect the beginning and ending narratively.

The Answer and Its Justification

The conceptual answer is: Maria had the key immediately after the transfer, under the literal physical-transfer interpretation. The justification identifies the event and rule used. It may mention that the identity of the person who left for the library is unnecessary to the conclusion.

A restricted formal certificate would verify the links among event, recipient, object, and time. A text-fidelity check would examine whether "Maria" really is the recipient and whether the event is conditional. These are two different checks. The second may require a linguistic evaluator or a human, depending on the controlled domain.

The final circuit may be small. The construction graph may retain alternatives and reasons why some were not explored. For ordinary auditing, we display the short justification. To investigate a failure, we must also be able to open the relevant construction trail.

Changing the Question

Now we ask: "Who left for the library?" The old possession interface is insufficient. The system returns to the richer representation and identifies two interpretations. In the absence of additional context, the correct answer is ambiguity.

Next we ask: "Who brought the key back to the laboratory?" The text does not specify. The guard finding it there on Tuesday does not identify who carried it. A generative component may propose scenarios, but these must be kept separate from the derived answer. "It does not follow from the text" is a useful conclusion about the limits of the information.

The question “could Maria have entered the laboratory on Monday evening?” requires additional rules: whether she had the key at that time, whether it opens the laboratory, whether access was permitted, and whether possessing the key is sufficient. A system answering merely because it recognizes the association between keys and doors may skip decisive conditions.

Correcting a Premise

We add: “Correction: Ana showed Maria the key, but did not give it to her.” This is not a monotone refinement of the old transfer; we withdraw that event's interpretation. The conclusion about Maria's possession loses its support. Independent information, such as the existence of the people or the key being found on Tuesday, may remain.

This is a strong test of the architecture. If the system retains the old answer because it appears in memory, provenance is not controlling acceptance. If it reconstructs the entire document without identifying the affected dependency, it may be correct, but has not demonstrated the advantage of local revision.

What We Learned from the Example

Operational understanding did not require choosing every detail of a complete world. It required a representation sufficient for the current question, continued access to unresolved distinctions, and the discipline not to fill gaps as facts. The same example could become a family of tests by changing the participants, time, verb, and question.

Success does not mean the system always says “Maria.” It means the answer changes exactly when a change in the text or question justifies it, and the explanation identifies that change. This distinguishes memorizing a pattern from using a structure.

21. How Circuits and Their Interfaces Might Be Learned

Three Different Objects of Learning

A system can learn parameters within a fixed operation. It can learn a new circuit by composing existing operations. It can learn a new representation in which many circuits become easier to construct. These three forms of learning should not be reported under the same label without differentiation.

For SOP Lang, the first may help recognize language. The second may produce reusable rules and procedures. The third is directly connected with sufficient interfaces. The ambition is for the system to discover that certain historical details no longer matter and that other relationships must be preserved.

A cautious path begins with researcher-provided interfaces, then asks the system to select among them. The next stage allows it to combine them. Only afterward do we ask it to invent new interfaces. Otherwise, total failure does not tell us whether the problem lies in selection, composition, or representation discovery.

Libraries That Grow from Solutions

If several problems use the same subcircuit, we can turn it into a reusable abstraction. Instead of searching for a long sequence of operations every time, we select a single higher-level concept. DreamCoder explores learning symbolic libraries together with neural guidance of search. [23]

Reuse may reduce search depth, but library growth increases the number of options. A library with thousands of nearly identical operations may be harder to search than a small one. Organization, contracts, and selection criteria must also be learned.

A useful abstraction must compress more than a single example. If we name every solved problem as a new operation, we have built a case memory. That can be useful, but does not demonstrate discovery of a general regularity. The test is reuse on combinations and instances absent during construction.

Description Length and the Price of Exceptions

The minimum description length principle suggests jointly evaluating model complexity and the cost of describing the data remaining after the model is used. [26] In library terms, we can balance the cost of new circuits against the cost of solutions expressed through them.

A rule that compresses a thousand cases and needs two exceptions may be valuable. A rule requiring an exception for every new case has not discovered much structure. Yet the shortest description is not automatically the true explanation. The choice of language, encoding, and objective affects the result.

For research, MDL can be a selection criterion, not a substitute for semantic validation. A very compact library may erase rare but essential distinctions. It must be evaluated on questions requiring them, not merely on average case frequency.

Learning a Merging Relation

The system may observe that two constructions behave identically on the current tests. It proposes a candidate shared interface. A verifier tries to find a continuation that distinguishes them. If it finds one, the interface is enriched. If a decidable theory exists for the relevant class, we can also obtain a proof. In other cases, we retain a bounded empirical guarantee.

Research on inferring rewrite rules using equality saturation is a precedent for discovering useful equalities in expression spaces. [31] Transferring this to document semantics, however, requires knowing what objects are being compared and which verifier can check their meaning.

Learning a vector in which two states are close is insufficient. Proximity can guide the search for an equivalence. Accepting an exact merger requires an additional argument. At an approximate level, we can allow errors, but they must be isolated from the level promising verified conclusions.

Is the System Learning, or the Researcher?

A transparent evaluation reports who supplied the primitives, semantic schema, examples, rules, and stopping criterion. If an external language model produces the entire circuit and SOP Lang executes it, the result may demonstrate interoperability and auditing. It does not yet demonstrate autonomous synthesis within the symbolic core.

If the researcher inspects every failure and manually adds the appropriate rule, progress is an assisted engineering process. That is not methodologically shameful. It becomes problematic only when the work is hidden and the result is presented as independent learning.

A mature stage would require the system to propose improvements, verify them within a declared framework, and accept them only when they reduce total cost or increase competence without uncontrolled losses. Until then, every intermediate level can produce useful scientific results.

22. Auditability: From a Complete Trace to a Usable Explanation

A Trace Is Not Yet an Explanation

A log of a million operations may enable execution to be reconstructed and still fail to help a researcher understand an error. Practical auditability requires selecting the relevant structure: decisive premises, transformations that change meaning, and the point at which an approximation was introduced.

For SOP Lang, we propose three views of the same result. The short view shows the main justification. The diagnostic view shows alternatives and checks that determined the choice. The reproduction view allows the relevant computation to be rerun. These are ways of presenting a trace, not reasons to invent a story different from the actual computation.

A system may produce a pedagogical explanation simpler than the actual path, but must label it a reconstructed explanation and show its verifiable connection to the result. It must not claim that the simplified story literally describes every internal step.

Value Provenance and Decision Provenance

Value provenance says where a conclusion comes from: sources, rules, computations. Search-decision provenance says why one circuit was tried before another. The first may suffice to verify the result. The second matters for understanding costs, selection errors, and possible systematic preferences.

A neural heuristic may rank candidates without being able to explain its scores intuitively. If subsequent verification is sound, this does not automatically compromise the result's formal validity. But it can affect which solutions are found within budget and which cases are abandoned. A performance audit must include this selection.

An “I did not find one” result may mean proven impossibility, exhaustion of a complete finite search, or exhaustion of the budget. These three situations must be presented differently. A verifier that accepts solutions is not automatically capable of certifying that no solution exists.

Revision and Local Invalidation

If a source is withdrawn, the system should identify conclusions depending on it and check whether they have alternative support. A shared provenance graph may allow this without reanalyzing every result. But sound reuse requires dependencies to be complete for the properties under consideration.

Caches must be indexed by the relevant context. Two questions with identical text but changed domain rules need not have the same answer. An interface ignoring an assumption's version may reuse an invalidated conclusion. This is a case of semantic insufficiency, not merely an administrative error.

For systems that act, the plan and the executed effect must be separated. An operation may propose sending a message without sending it. Verifying a plan does not authorize all its effects. These distinctions do not dominate the book's theory, but become necessary when circuits move beyond pure computation.

How We Measure Auditability

We can construct diagnostic tasks involving a wrong premise, a wrong rule, or an unjustified merger. Researchers who did not build the system receive the explanations and must locate the problem. We measure time, the proportion of correct diagnoses, and the number of details they need to consult.

We can compare the short explanation with the raw log and with a system lacking provenance. If explicit structure does not reduce diagnostic cost, the promise of intelligibility remains unproven. If it reduces that cost only for the system's authors, interfaces for external users must be improved.

Auditability may conflict with confidentiality or data volume. An explanation may reveal sensitive information even when the final answer is public. The project must specify what each evaluator can see and which guarantees can be checked through more restricted certificates. Transparent need not mean indiscriminately accessible to everyone.

23. What Controlled Models Indicate and What They Do Not

Why We Need Small Models

A small model can isolate the mechanism we advocate. To find out whether semantic merging reduces search, we do not initially need every difficulty of natural language. We need a family in which equivalence is exact and results can be compared with an independent method.

The observations presented here come from such finite models, rerun and checked. They justify interest in the mechanisms tested. They do not demonstrate a general machine of understanding or validate every component of the proposed architecture.

Exact Merging of Functional Constructions

In a family of finite arithmetic circuits, each stage permits two transformations. With n stages, there are 2^n choice sequences. Nevertheless, the transformations preserve an affine function form, f(x) = ax + b, with arithmetic modulo 31. Coefficient a remains nonzero, so there are at most 30 × 31 = 930 functions of this form.

The coefficient pair, together with the stage, allows exact merging of constructions with the same functional behavior. At 16 stages, the 65 536 possible sequences were represented by 430 distinct final functions; forward exploration evaluated 4 794 extensions. At 40 stages, the number of possible sequences is, by formula, 1 099 511 627 776, while forward exploration evaluated 41 110 extensions and reached 930 distinct final functions.

At 80 stages, the number of possible sequences is approximately 1.21 × 10^24, while the number of forward extensions evaluated was 115 510. This contrast shows the difference between the number of histories and the number of relevant functional states. The enormous sequence counts are calculated using the combinatorial formula; they were not obtained by actually enumerating every sequence.

What Was Independently Verified

For sizes up to 16 stages, an independent enumerator verified the functions and the multiplicity of constructions producing them. For larger cases, final representations were checked on all 31 domain inputs, and a circuit satisfying the target function was reconstructed. Functions were not declared equal on the basis of a single input example.

The numbers above concern forward extensions, not the total cost of the entire procedure. There were also backward checks, function checks, storage, and counting operations. We do not present these values as timing measurements or speedup factors against an optimized competitor.

The affine representation was supplied, not discovered. Modulus 31 was small and fixed. If the modulus becomes m, the number of states may grow on the order of m^2; this does not automatically mean poly-

nomial cost in the number of bits needed to write m. Arbitrary topologies, linguistic interpretation, and autonomous interface learning were not tested.

A Counterexample to Merging on Too Few Tests

In the same kind of domain, two functions can coincide on input zero and differ on input one. This simple control confirms that a shared answer on one example does not justify universal merging. Without additional verification, an empirical signature is at most a candidate filter.

The positive result and negative control must be read together. Compression alone is not enough. We must also show that a representation does not compress two states that ought to remain distinct. Perfect compression identifying every circuit with a single state would have an excellent size and almost no semantic usefulness.

Relations Detect a Conflict That Individual Domains Leave Undecided

Binary systems of equalities and inequalities expressed through addition modulo two were also analyzed. In cycles with an incompatible final parity, checking local value supports eliminated no individual value. The correct result for that procedure was “undecided.” Preserving relations and applying linear elimination produced an explicit contradiction certificate.

Consistent and inconsistent cycles up to 256 variables were checked. Small cases, up to 16 variables, were also checked through complete enumeration. Consistent cases of this form had two solutions, while in inconsistent cases combining the equations produced 0 = 1.

This does not demonstrate that all local propagation fails, or that only a sophisticated global method can solve the problem. The cycles have a simple structure; methods preserving parity links can solve them efficiently. The result isolates the weakness of exclusively individual representations and the value of an appropriate relational algebra.

A Common Interpretation of the Observations

The two result families support the same direction from opposite angles. In the first, an exact summary merges irrelevant histories and reduces search. In the second, an impoverished summary loses the dependency that would enable a conclusion. There is no contradiction: forget what no longer matters and preserve what a continuation can use.

What remains to be demonstrated is far more ambitious: that such representations can be found for useful language tasks, that they can be learned, that verifying them remains economical, and that the advantage persists as domains, questions, and sources vary. The finite models justify exploring these questions; they do not already provide their answers.

PART VI

The Research Program

Testable hypotheses, families of experiments, refutation criteria, and stages for future researchers.

24. Hypotheses That Can Be Refuted

From a Vision to Claims That Risk Failure

“Circuits will make intelligence clearer” is too broad an aspiration to test directly. It must be broken down into claims that can fail separately. A negative result on one must not be concealed by success on another. Speed, correctness, generalization, and auditability are different objectives.

We will consider a hypothesis useful when it specifies the domain, mechanism, comparison, and observation that would put it under pressure. We do not require every hypothesis to have a single universal number. We require thresholds and measurements to be established before selecting favorable examples.

H1 — Semantic Merging Reduces Search Without Changing Solutions

In a finite problem family, an interface declared exact should preserve the set of answers obtained by exhaustive enumeration and reduce the number of extension evaluations when many equivalent histories exist. The baseline must not be deliberately weak: include memoization by syntactic identity, not just naive enumeration.

A single case in which a valid answer disappears or an invalid one appears refutes the exactness of the implementation or interface for the declared class. A lack of cost improvement does not refute exactness, but does refute practical advantage in the tested regime. The two results are reported separately.

The expected phenomenon is saturation in the number of distinct states when the accessible semantic space is small, while the number of histories keeps growing. If the number of states nearly tracks the number of histories, there is insufficient reuse for the proposed mechanism.

H2 — Appropriate Relations Remove Bottlenecks of Individual Summaries

We compare the same problems using independent variable domains and representations preserving a class of relations. The hypothesis is not that relations are always cheaper. It is that, in families where an omitted dependency is decisive, the additional cost of the relation may avoid much more expensive branching.

We measure detected conflicts, unresolved cases, memory and total cost. An interesting negative result occurs when the relational domain grows so large that the advantage disappears. This suggests selective use of relations or choosing another algebra, rather than necessarily abandoning the entire direction.

H3 — Ambiguity can be preserved selectively, without premature choices

On texts with controlled interpretations, the system should correctly answer questions whose answers remain invariant across interpretations and preserve uncertainty when the answers differ. The comparison includes a system that immediately chooses the most likely interpretation and one that enumerates all complete interpretations.

Success means reducing cost relative to enumeration without increasing unsupported conclusions, and making fewer errors than premature selection on ambiguous cases. If the system appears to answer well only because the question set avoids all relevant ambiguities, the test does not support the general hypothesis.

H4 — Interfaces can be learned and reused

Interfaces discovered on one family of problems should reduce cost on new instances or new compositions without requiring an exception for every case. The comparison includes expert-provided interfaces, generic interfaces and the absence of semantic compression.

Learning cost, the number of counterexamples and validation work must be included. An interface that saves one second on a single query after a week of searching may be an interesting theoretical result, but offers no practical advantage in that regime. For thousands of queries, the same investment may have a different value.

H5 — Verification and selection can usefully be separated

A learned strategy should find good candidates faster, while verification preserves the stated guarantee. We compare simple deterministic orderings, manually designed heuristics and learned proposals, using the same verifier and budget.

If accuracy appears better only because verification is more permissive, the hypothesis is not confirmed. If the learned strategy systematically misses rare solutions within the budget, that effect must be reported even when the average improves. The exploration policy can introduce a structural preference for frequent cases.

H6 — Structural traces reduce diagnostic cost

Circuit-based explanations should help independent evaluators locate an error faster and more accurately than explanations without provenance or raw logs. We measure diagnostic time and quality, not just reported satisfaction with the text.

A failure here would matter: the system may be formally verifiable yet still difficult for people to understand. Research should then address how explanations are presented, without claiming that the mere existence of the circuit solves intelligibility.

An open philosophical hypothesis

The broader hypothesis is that discovering relevant distinctions is a central component of understanding. Experiments can support this idea through converging results, but cannot immediately turn it into an exhaustive definition of the mind. Social skills, goals, embodiment, experience and context may require additional dimensions.

The program's value does not depend on monopolizing the word “understanding.” If we obtain systems that generalize better, fail more visibly and can be corrected more precisely, the result matters even under a cautious philosophical interpretation.

25. The formal laboratory: which simulations should be built

Family A — Small semantic states, vast numbers of histories

We begin with transformations that preserve a known family: finite affine functions, permutations of a small domain, automata for string properties or operations on finite sets. We vary construction length, domain size and the number of available operations separately.

For each family, the researcher defines the exact interface and verifies the composition property. On small cases, an independent enumerator establishes the ground truth. On larger cases, we check witnesses, the interface invariant and, where possible, formal certificates. We do not use the same mechanism to produce and confirm every answer.

We seek three regimes: strong compression, moderate advantage and almost no compression. All must be published. A result showing only the favorable regime does not tell us where the method applies.

Family B — Representations that are too poor, appropriate or too rich

We take the same problem class and change only the interface. One variant preserves individual values. Another adds equalities. Another preserves linear equations or more general relations. An extreme variant preserves the entire concrete state.

The phenomenon sought is a trade-off: an overly poor interface produces many unresolved cases or, if incorrectly treated as exact, errors; an appropriate interface reduces search; an overly rich interface may incur high storage and normalization costs. We do not assume in advance that a single minimum exists or that it remains stable across families.

A useful result would be an adaptive selection rule: start with an inexpensive representation and add a relation when a counterexample shows it is necessary. This must be compared with choosing a rich representation from the outset, including the cost of every intermediate refinement.

Family C — Topology and algebra vary independently

We generate constraint graphs with low or high connectivity and relations with or without exploitable algebraic structure. A cycle of simple binary relations does not pose the same kind of difficulty as a dense graph of arbitrary constraints. A dense linear matrix may be easier to solve than a sparse family of nonlinear relations.

The experiment must avoid confusing graph size with semantic difficulty. We can keep the number of variables fixed and change the relations, then keep the algebra fixed and change the topology. We measure frontier size, elimination cost, the number of branches and peak memory.

A promising phenomenon would be automatic recognition of a linear subcircuit or a conditional independence, followed by the use of a specialized interpreter. This would illustrate how knowledge of structure changes the algorithm, rather than merely the order of attempts.

Family D — Learned equivalences and adversarial counterexamples

We start with empirical signatures built from a few probes. We ask the system to find states that the signature merges but a continuation distinguishes. We then observe whether refinement stabilizes into a small representation or keeps accumulating exceptions.

Counterexamples must be sought deliberately, rather than awaited by chance. We can vary boundary values, operation order, the numeric domain and preconditions. For languages with partial operations, we also test inadmissible inputs. A rewrite that is correct for ideal numbers may fail under semantics with overflow or special values.

We report proven equalities, equalities exhaustively checked over finite domains and equivalences supported only empirically separately. These categories can coexist in a library, but must not share the same certainty label.

Family E — Memory, recomposition and amortized cost

An interface can gain value through reuse. We construct sequences of questions over the same knowledge base, then introduce local changes to the data or rules. We compare complete recompilation, provenance-based updating and rebuilding views from the persistent source.

Total cost can be expressed conceptually as the sum of learning, compilation, query, verification and update costs. Reporting only the latency of an already prepared query is insufficient. Reporting only the initial cost likewise fails to capture the value of a reused library.

An interesting result would be a reuse threshold beyond which the learned interface becomes economical. Another would be finding that frequent changes destroy the advantage of compilation. These results would guide the choice of suitable applications.

The minimum protocol

Problem sizes and distributions must be specified before interpreting the results. We use multiple instances and, when generation or selection is random, multiple initializations. We report dispersion, not just the mean. For deterministic procedures, we vary problem families and ordering, because determinism does not eliminate sensitivity to input.

We retain solved cases as well as failures, exhausted budgets and excessive resource consumption. A correct result obtained at enormous cost is not the same as a feasible result. A fast but incorrect result must not be hidden in an average speed figure.

26. The language laboratory: from controlled texts to real documents

Why we begin with a controlled language

To evaluate reasoning, we must know whether the input interpretation is correct. In fully open natural-language texts, failure may arise from vocabulary, pragmatics, missing knowledge or synthesis. A controlled language allows these components to be isolated without pretending to encompass all of natural language already.

In the first stage, the researcher generates structured situations and corresponding texts. The situation supplies the ground truth. Tests include paraphrases, not just a single template. Later, human texts are annotated independently, with legitimate disagreements marked.

A well-organized test separates three conditions: reasoning over correct representations supplied directly, interpretation evaluated separately, and the complete system from text to answer. Differences between results show where the difficulty lies. A planning error must not be attributed to the parser, or vice versa.

A curriculum of phenomena

We begin with identity, membership and simple transfers. We then add temporal ordering and persistence. Coreference, alternatives, explicit negation, quantifiers and conditionals follow. Only once these are controlled do we introduce belief reports, causal explanations and variable background knowledge.

Each phenomenon has minimal pairs of texts: we change a single element that should change the answer. “All” becomes “some”; “before” becomes “after”; “gave” becomes “showed”; “says that” is added before a statement. There are also changes that should not affect the answer: consistently renaming people or adding an independent detail.

Evaluation must include new compositions of phenomena, not just new examples from the same templates. A system that handles negation and time separately may fail when negation concerns an event within an interval. Compositional generalization must be tested explicitly.

Questions that require retaining rare distinctions

The test set must include questions about exceptions, not just the dominant conclusion. A representation that smooths over rare differences may produce pleasing summaries and wrong answers precisely in important cases. This is a problem both for statistical models and for overly general symbolic rules.

We can construct documents in which nine events follow a rule and the tenth introduces an explicit exception. We ask first about the trend, then about the exception. A system sufficient for the first question must not claim sufficiency for the second without returning to the retained information.

We thus measure the cost of changing the objective: how many details must be recovered, how long refinement takes and whether the old answer is withdrawn when it no longer matches the question. This directly tests the separation between persistent memory and the task-specific view.

Proofs and benchmarks

ProofWriter and PRover are reference points for inference and proof-generation tasks using linguistically expressed rules. They show the usefulness of evaluating conclusions and justifications together within bounded domains. [28, 29] They must not be treated as exhaustive evidence of open-ended understanding.

For SOP Lang, we can adapt this idea without reducing everything to a single benchmark score. We evaluate answer correctness, step validity, fidelity to sources, the capacity to abstain and responses to changed premises. A correct answer with an invalid proof is a separate category from a correct, justified answer.

When a dataset contains genuine ambiguities, we do not artificially force a single interpretation to count as absolute truth. We can assess whether the system preserves acceptable interpretations and whether its answer correctly expresses what they share or where they diverge.

Comparison with language models and hybrid systems

Comparisons must hold available information, tool access, budget and the success criterion fixed. A symbolic system given a perfect interpretation is not directly comparable to a model given only raw text. Both conditions are useful, but answer different questions.

We test a language model used directly, a circuit system with supplied rules, a system with learned selection and a hybrid system with verification. We hold the verifier constant where we want to isolate the effect of search strategy. We also report cases where a language model solves something the symbolic library cannot express.

Average accuracy must be accompanied by coverage and abstention. A system answering very few questions can have excellent precision without broad competence. A system answering everything can have good coverage and make many unsupported claims. The curve relating coverage to error is more informative than a single number.

Phenomena worth tracking

A promising phenomenon would be almost linear cost growth for many independent ambiguities, followed by a sharp increase when those same ambiguities become globally correlated. This would show that sharing exploits genuine independence, rather than merely template peculiarities.

Another phenomenon would be transferring a learned possession circuit to a domain with a similar structure but different vocabulary. A negative result would be needing retraining or new rules for every superficial rephrasing. In real documents, we may also observe a knowledge limit: the circuit is correct, but the necessary premise appears nowhere. The system must acknowledge the absence, rather than mask it with fluency.

27. The strongest objections

“It is just dynamic programming under another name”

Part of the mechanism is indeed dynamic programming over an appropriate semantic state. Claiming this principle as novel would be dishonest. The possible contribution lies in the representations chosen for linguistic knowledge, the composition of guarantees and the learning of interfaces that make sharing possible.

If the project produces only a new notation for known methods, its value may be in engineering or teaching, rather than a new theory of intelligence. That assessment must be accepted. Excellent integration can be useful without an exaggerated mathematical claim.

"All the meaning has been hidden in the primitives"

The objection is valid when basic operations already solve the difficult problems and the circuit merely connects them nominally. The response must analyze each component's contribution. What does the linguistic interpreter do? What does the verifier do? What is supplied manually? What is learned?

An ablation test removes or replaces a component to see which competence disappears. If all performance comes from an external model, the project may remain useful as an auditing environment, but has not demonstrated an autonomous reasoning alternative. An intermediate result must not be stretched beyond what it shows.

"The sufficient interface is as large as the problem itself"

In some classes, that may be the correct answer. An arbitrary future question may require any detail of the past. Unless we restrict continuations or accept losses, compression may be small or nonexistent. This limit must not be hidden by choosing a convenient family of questions after observing the results.

The program must identify classes with reusable structure and those without it. Useful systems may employ local, temporary interfaces, rather than a single global essence of a document. A negative finding about universal compression can lead to a better memory architecture.

"Verifying meaning is as hard as understanding"

For open-ended language, this is often a serious objection. A verifier cannot decide every interpretation exactly without a sufficient model of language and the world. Separating proposal from verification does not supply an omniscient judge for free.

Nevertheless, partial checks can be valuable: the source contains the quotation; an event's roles are consistent; the conclusion follows the stated rules; time has not been confused; a counterexample invalidates the generalization. Verification coverage must be reported. A collection of local guarantees must not be called certified global truth.

"Probabilities reintroduce precisely the opacity we wanted to avoid"

Probabilities can describe legitimate uncertainty, and learned strategies can reduce search. The problem is not their presence but the mixing of roles. A score can rank candidates without establishing a candidate's truth. A distribution can describe alternatives without turning them into certainties.

Opacity remains a problem for diagnosis and fairness in selection. We can limit the role of opaque components, compare heuristics and retain verification wherever specifications exist. It does not follow that every part of the system will become intuitively transparent to everyone.

"A formally correct system can be practically useless"

Yes. It may answer only trivial questions, incur high modeling costs or require so many clarifications that the user gives up. A research program must include task usefulness and the total cost of use, not just the beauty of its guarantees.

Initial domains should be chosen where a wrong conclusion has a cost, sources are accessible and rules can be specified: certain forms of document analysis, requirements tracking, consistency checking and reasoning about procedures. This is a proposed research choice, not a promise that every such application has already been solved.

"An elegant explanation can become a new kind of illusion"

A well-drawn circuit inspires trust. Precisely for that reason, fragile premises, missing checks and unresolved alternatives must be shown. Formalization can create an illusion of rigor when the reader cannot see how much informal interpretation precedes the first node.

The answer is not to abandon formalization, but to extend the audit to the boundary between source and model. A machine of understanding should make that boundary more visible than it is in an ordinary fluent answer. If it hides it, it has missed one of the project's central motivations.

28. Stages of an open program and criteria for continuing

Stage one: a shared vocabulary and verifiable models

The first necessary result is explicit semantics for a small class of circuits: what values, alternatives, relations and guarantees are. Finite tests are built with accessible ground truth, and the separation between execution, synthesis and audit is verified.

The criterion for advancing is not a demonstration of general intelligence. It is the absence of semantic errors within the exact class declared, independent reproduction of results and identification of actual costs. Any failure of exactness must be explained before the architecture is extended on an unstable foundation.

Stage two: interfaces selected and refined automatically

The system receives a library of abstractions and must choose the appropriate interface for a task. It is then allowed to combine abstractions and refine them in response to counterexamples. Selection cost and the cost of representation changes are measured.

Advancement is justified if adaptation yields gains on families not used for tuning and does not conceal semantic losses. If selection costs more than it saves in every regime tested, the strategy must be revised or the domain narrowed. An adaptive mechanism is not useful merely because it seems more intelligent.

Stage three: controlled language and revisable memory

Circuits are connected to a language rich enough for identity, events, time, negation and ambiguity. Changes of question and source corrections are introduced. We separate interpretation errors from reasoning errors.

An important condition is the ability to recover a new answer from the preserved source when the previous interface was insufficient. Another is withdrawing conclusions after premises are corrected. Without these behaviors, the system risks being merely a solver of static problems, rather than an evolving knowledge memory.

Stage four: learning circuits and representations

The system proposes new abstractions, compares them with the library and attempts to validate them. The researcher need not be eliminated, but their role must be accounted for. Both accepted and rejected proposals are preserved, together with the reasons.

The essential criterion is transfer: a new representation must help with problems not used to invent it. If progress consists only of memorizing solutions, we name the result accurately and continue seeking the generalization mechanism. We do not rename memory as understanding to save the hypothesis.

Stage five: real documents and independent researchers

Only after the basic mechanisms stabilize do we evaluate real documents with ambiguities, incomplete sources and disagreements. Researchers who did not build the system participate. Tasks are chosen for usefulness and the possibility of evaluating errors, not merely to produce demonstrations.

A mature result would include performance, cost, verification coverage, abstention cases and diagnostic cost. It would show where the system is preferable to an alternative and where it is not. A contribution that rigorously delineates a domain of usefulness can be more valuable than a spectacular but unevaluable demonstration.

The roles of human and AI researchers

The human researcher can contribute intuitions, goals, judgments about relevance and interpretations of failures. An AI partner can propose formalizations, examples, counterexamples and architectural variants. Neither should be treated as an infallible verifier of their own proposals.

Sound practice separates generation and criticism, uses independent verifiers where possible and preserves methodological decisions. A claim supported by multiple responses from the same type of model does not automatically acquire independent confirmation. Independence must concern the mechanism and source of evidence, not merely the number of texts produced.

What we retain from the initial intuition

We retain the idea that a circuit representation can unify data, knowledge and transformations in an inspectable object. We retain the possibility that intermediate representations determine how difficult it is to construct reasoning. We retain the ambition to learn circuits and interfaces, rather than merely execute what was written manually.

We abandon the assumptions that an explicit circuit is automatically easy to find, that a well-typed wire preserves every relation or that a formal proof solves the interpretation of the world. These concessions do not empty the project of substance. They identify its central question.

We seek more than a language for describing reasoning. We seek representations in which different lines of reasoning can share their future without falsifying their meaning.

If this direction succeeds, part of the progress will not appear as an ever larger circuit. It will appear as the discovery of a smaller, more appropriate frontier: rich enough for the questions that matter, disciplined enough to support continuation and explicit enough to be challenged.

If this direction fails in certain classes, a well-documented result will show which distinctions cannot be compressed into the proposed form and where the cost moves. This is what an open research program means: not protecting an intuition from evidence, but constructing the evidence that can transform it into better knowledge.

The reader's workshop

Step-by-step arguments, exercises with answers and a shared vocabulary for continuing the research.

A. Three arguments worth understanding fully

A.1. Why an exact interface allows histories to be merged

Suppose two states s and t have the same summary: α(s) = α(t). Suppose every admissible operation F has a version F# on summaries and α(F(s)) = F#(α(s)). Further suppose the relevant result R can be obtained through a function h of the summary.

We apply the same operation F to both states. The resulting summaries are F#(α(s)) and F#(α(t)). They are equal because the arguments are equal and the operation F# is the same. We can repeat the argument for the next operation. After any finite number of steps, the summaries remain equal. The observable results are then equal, because both are calculated through h from the same summary.

This proves equivalence for continuations composed of the declared operations. If the operation-selection policy consults a part of the state absent from the summary, the argument does not automatically cover that policy. We must either include the information consulted or define continuations as fixed external sequences and analyze the selection strategy separately.

In a system with partial operations, admissibility must also be preserved: two merged states cannot permit an operation in only one of them if that difference matters. In a nondeterministic system, sets or distributions of outcomes must be compared according to the adopted semantics. The simple form above concerns deterministic transitions compatible with the summary.

What the argument does not prove is the existence of a small summary that is inexpensive to compute or easy to learn. The property can hold with α equal to the identity, preserving the entire state. This is exact and may compress nothing. The research gains value when we find interfaces much smaller than the histories they represent.

A.2. How an approximation remains sound

Let C be the set of concrete states still possible and a an abstract description containing them: C is included in γ(a). Suppose the abstract transformation F# is sound, meaning that all results obtained by applying F to any state in γ(a) are included in γ(F#(a)).

Because C is included in γ(a), the results F(C) are included in F(γ(a)). By the soundness assumption, that latter set is included in γ(F#(a)). Therefore, the new abstract description includes all actual concrete results. The argument repeats at every step.

We can now understand what successful verification means. If every state in the abstract description satisfies the required property, every concrete state also satisfies it. If the abstract description admits a bad case, that case is not necessarily real. It may be a false alarm introduced by approximation.

When searching for a solution, a nonempty abstract intersection with the goal is insufficient. The candidate must be concretely realized or checked in a more precise representation. Conversely, if a sound overapproximation of the results does not intersect the goal at all, we can reject the branch, because the actual results cannot reach it either.

This is why abstractions are very good for filtering but require caution when accepting results. The same representation can be sound for saying “impossible” and insufficient for saying “found it,” depending on the logical direction of the test.

A.3. Why individual summaries do not preserve every relation

Let relation R consist of the pairs (0, 0) and (1, 1). Its projection onto the first variable is {0, 1}; its projection onto the second is also {0, 1}. If we reconstruct the possibilities using only these two projections, we obtain their product: four pairs.

Two pairs, (0, 1) and (1, 0), did not belong to the original relation. The operation “preserve each component separately” lost the equality dependency. It lost no individually possible value. Precisely for that reason, a check examining only each wire's domain can miss the problem.

Reconstruction through the product is a sound overapproximation of R. It can be used for certain tests, but not as though it were the exact relation. If a continuation asks whether x = y, R always answers yes, while the product also admits a negative answer.

A compact repair is to preserve both domains and the constraint x = y. Enumerating every pair was unnecessary. For other relations, an equally small repair may not exist in the chosen language. Research into interfaces is therefore also research into a vocabulary of useful relations.

B. Conceptual exercises with explained answers

1. A summary for the mean

A machine reads numbers one at a time. It preserves only their sum and must report their mean at the end. Is the interface sufficient?

No. The lists [2, 4] and [6] have the same sum but different means. The element count is the missing distinction. The pair “sum, count” can be updated compositionally when an element is added and allows the mean to be calculated for nonempty lists. The empty-list case must be handled explicitly, rather than hidden in an invalid division.

2. The same mean, a different question

Suppose we preserve the sum and count. Can we decide whether every element of the list was equal?

Not in general. The lists [1, 3] and [2, 2] have the same sum and element count. We can add a suitable representation, such as a reference value and a flag indicating whether every element equals it. The exercise shows that a sufficient interface is not a universal essence of the data.

3. Correct type, wrong meaning

An operation receives two numbers and adds them. The first is a price in euros, the second a temperature in degrees Celsius. The system checks only the numeric type. What is missing?

The semantic type or contract specifying which quantities can be combined is missing. Checking executability does not establish the operation's relevance. The solution is not to prohibit general arithmetic, but to distinguish the raw data level from justified use within a model.

4. A nonempty intersection

The analysis says an operation can produce any number between 0 and 10. The goal is 7. Can we declare that an execution producing 7 exists?

No, if the interval is only an overapproximation. The actual results might be only 0 and 10. The intersection shows abstract compatibility, not realizability. A concrete witness must be checked, or the representation refined until existence is justified.

5. A contradiction between sources

One document states that Ana is in the laboratory at 10 o'clock. Another states that she is not in the laboratory at the same time. What should the system produce?

It must first establish whether they refer to the same person, place and interpretation of the time. If so, and if both statements are accepted as premises about the same world, the conflict must be flagged. It can separately preserve the fact that each source made its statement. It must neither automatically choose the preferred source nor turn incompatibility into permission to answer anything.

6. A rule that forgets a precondition

A circuit rewrites x/x as 1. What must be checked before adding the rule to an equivalence class?

The number semantics and domain of definition must be specified. In ordinary arithmetic, the condition that x is nonzero is essential. Particular numeric semantics may introduce other special cases. A rule correct under a precondition cannot be applied unconditionally merely because it simplifies the expression.

7. An irrelevant ambiguity

“Ana gives Maria the key. She leaves.” The question is who has the key immediately after the transfer. Must the pronoun's referent be chosen?

No, in the restricted model where the transfer is actual and complete and departure does not change possession at the time asked about. Both interpretations yield Maria. If the question becomes who leaves, the ambiguity must be preserved or resolved through additional information. Relevance is relative to the continuation.

8. An explanation with two dependent sources

Two articles support the same claim, but both copy a single report. Can we count this as two independent confirmations?

Not from the number of articles alone. Provenance reveals their common dependency. We may have two reproductions of the same source, rather than two independent observations. A system can count derivations, documentary sources and information origins, but these are different quantities.

9. An ambiguous negative result

The expander finds no circuit before its budget runs out. What can it claim?

It can state that it found no solution within the explored space and budget. It cannot declare impossibility without further justification. If the library lacks the necessary operations, the absence of a solution says something about the search language, rather than the task's impossibility in general.

10. Compression that erases provenance

Two circuits produce the same conclusion, one from an accepted source and the other from an unverified hypothesis. Can we preserve only one, chosen arbitrarily?

Only if provenance affects no relevant continuation and the requested result requires no particular support. In epistemic applications, this condition is often false. We can merge the value while retaining distinct justifications, or include source status in the interface. Value equivalence is not automatically epistemic equivalence.

11. A vector representation exact on the samples

A vector allows every role in a set of one hundred examples to be recovered correctly. What conclusion is justified?

That the representation and recovery procedure succeeded on those examples under the tested conditions. Neither universal exactness nor stability as the number of relations grows follows. Subsequent tests should vary superposition, object similarity, roles and memory size, including adversarial cases.

12. An abstraction that grows without stopping

Every counterexample prompts the system to add a special case. After a thousand examples, the representation contains almost a thousand exceptions. Is this the learning of a useful sufficient interface?

It may be an exact memory of the cases, but does not demonstrate useful structural compression. It must be compared with direct memorization and evaluated on new cases. A negative result may suggest that the abstraction language is unsuitable or that the problem family lacks the regularity sought. It must not be hidden by renaming exceptions as “concepts.”

13. Prediction or intervention

A system predicts that people with wet umbrellas have been in the rain. Can it infer that wetting an umbrella will cause rain?

No. The first relation is predictive within the observation model. The second concerns an intervention in the world. A model of causal mechanisms is required. The graph through which the system computes its prediction is not automatically the phenomenon's causal graph.

14. A gain that disappears after accounting

A system answers ten times faster after expensive compilation. How do we decide whether it is more efficient?

We specify the number of queries and the update frequency. We add compilation, verification, memory and answer costs. We may obtain a reuse threshold beyond which the method becomes advantageous. Without specifying that usage regime, the “faster” comparison is incomplete.

15. What deserves to be called progress in understanding

One system answers a question correctly, then changes its answer after a rephrasing that leaves the meaning unchanged. Another abstains on the first question, explains the ambiguity and answers after a relevant clarification. Which result is more informative?

It depends on the task, but the second provides clearer evidence of control over missing information. The first isolated success may be accidental. Evaluating understanding must track families of variations and responses to them, rather than a single lucky answer. The ability to locate an insufficient representation is part of the competence sought.

C. Working glossary

Abstraction. A representation that preserves certain properties of a state and omits others. Its value depends on the operations and questions pursued.

Abstention. An explicit decision not to issue an unjustified conclusion. It must be distinguished from technical failure and proven impossibility.

Admissibility. The condition under which an operation can legitimately be applied, rather than merely mechanically executed.

Relational algebra. A family of operations on relations, such as selecting and combining tables. More broadly, the book uses the algebra of relations for combination rules that preserve dependencies.

Alternative. A possible case that must not be arbitrarily combined with consequences of an incompatible case.

Auditability. The ability to examine a result's premises, transformations and limits. Practical auditability includes the cost of that examination.

Trusted base. The components and assumptions on which the validity of the entire system's verification depends.

Bisimulation. A relation between states that can mutually match one another's observable steps in a transition model.

CEGAR. Refining an abstract model after discovering that an apparent counterexample is not concretely realizable.

CEGIS. Improving a candidate program using inputs on which it violates the specification.

Certificate. An object that allows a claim to be verified within a declared model. It does not automatically certify the external truth of the premises.

Circuit. A structure of operations and dependencies between values. In this book, the term does not require electronic components or binary values.

Knowledge compilation. Transforming a representation to make certain queries or operations more efficient, at an explicit upfront cost.

Compositionality. The ability to derive the whole's behavior from its components and the rules connecting them.

Congruence. Equivalence preserved by relevant operations and contexts. It allows a component to be replaced without changing the observation of interest.

Materialized content. The value produced by execution, such as an event table; it is not the same as the circuit source.

Semantic contract. A specification of the relation between an operation's input and output, including necessary conditions.

Counterexample. A concrete case that violates a general claim or distinguishes two unjustifiably merged states.

Correlation. A dependency between values that cannot always be reconstructed from descriptions of each value separately.

Abstract domain. The space of descriptions used by an analysis, together with the relevant operations and ordering.

E-graph. A shared structure for expressions merged through accepted equalities and their compatibility with context.

Contextual equivalence. The impossibility of distinguishing two constructions through admissible continuations and observations.

Epoch. A conceptual interval of computation with certain versions and assumptions fixed; changes can open a new epoch.

Expander. The proposed component that develops and selects circuit continuations relative to a goal and available information.

Frontier. The collection of values and relations through which a partial construction can interact with future operations.

Exact guarantee. Preservation of the stated property without loss, within the specified domain and premises.

Sufficient interface. A summary preserving everything required by relevant continuations and observations within the class analyzed.

Abstract interpretation. Systematic computation with descriptions of states, usually to obtain sound approximations of behavior.

Lattice. An ordered structure in which descriptions can be combined through operations corresponding to common bounds.

Witness. A concrete example demonstrating the existence of a solution, such as a circuit and its verifiable execution.

Monotonicity. Preserving an order when applying an operation. In information propagation, it enables reasoning about stable refinement.

Precondition. A requirement on the input or context that makes an operation applicable or guarantees an output property.

Provenance. The structure of sources and transformations on which a value or conclusion depends.

Fixed point. A state in which reapplying the operation or rules under consideration no longer changes the description.

Refinement. Introducing a distinction or narrowing the possibilities. It must be distinguished from revising a withdrawn premise.

Semiring. A structure with two operations used for alternatives and composition, under algebraic laws suited to the intended computation.

SSA. A representation with a single assignment location for each static result name; it does not exclude program loops.

Overapproximation. A description that includes all real possibilities and may include additional ones.

Treewidth. A measure of a graph's structural width, relevant to certain decompositions and algorithms; it does not describe all difficulty on its own.

Question-specific view. A temporary representation optimized for a goal, derived from a richer memory or source.

VSA/HDC. Families of high-dimensional distributed representations and operations for encoding, composition and retrieval.

Widening. A technique for accelerating an analysis's stabilization through a broader approximation, trading off precision.

D. A publishable result record

The claim and its domain

Every result of this program should begin with a bounded claim: which circuit family, operations, representation and questions are covered. Phrases such as “understands text” or “eliminates complexity” must be replaced with observable competencies and conditions.

What was supplied and what was learned

The rules written by the researcher, data schemas, external components, predefined interfaces and learned contributions are specified. The cost of building the library and verifiers does not disappear from the explanation. Future researchers must be able to identify the contribution's exact location.

Proof and verification

Mathematical proofs, exhaustive finite checks, empirical tests and exploratory observations are distinguished. Numerical results indicate what was measured and what was calculated by formula. An independent check must reduce the risk of repeating the same error, rather than merely rerun the same procedure.

The complete cost

Learning, compilation, search, verification, memory and updating are reported. The number of expansions evaluated is useful, but does not substitute for time when expansion costs differ greatly. Input size must also be expressed in bits when numerical values grow.

Cases that did not work

Failures, inconsistencies, unresolved cases and budget overruns are published. The report identifies whether the problem lies in interpretation, representation, selection, verification or expression. A well-localized negative result is more useful than a general promise without a diagnosis.

What would change the conclusion

The report ends with the evidence that would invalidate its claim and the experiments needed to extend it. This section is not a formality. It gives human and AI researchers a concrete way to continue without uncritically inheriting the authors' enthusiasm.

Annotated bibliography

Primary references and official documentation

Titles link to publications or official pages. The notes indicate each source's role in the research program. The publication year is distinguished from the preprint year where that difference is relevant. Online sources accessed: September 11, 2026.

[1] Cytron, R.; Ferrante, J.; Rosen, B. K.; Wegman, M. N.; Zadeck, F. K. (1991). Efficiently Computing Static Single Assignment Form and the Control Dependence Graph. ACM Transactions on Programming Languages and Systems.

The historical foundation of SSA in compilers. Relevant to stable result names and control dependencies; it does not by itself provide semantics for understanding.

[2] LLVM Project (online documentation). LLVM Language Reference Manual. Official documentation; accessed September 11, 2026.

A technical reference for SSA representation, control, types and operations. It helps separate the assignment convention from additional assumptions about purity, memory or loops.

[3] Fong, B.; Spivak, D. I. (2018). Seven Sketches in Compositionality: An Invitation to Applied Category Theory. arXiv:1803.05316; subsequently published as a volume in 2019.

An introduction to the mathematics of composable systems. It supplies vocabulary for relations, processes and structure-preserving translations. This book uses the idea of composition without assuming familiarity with the formalism.

[4] Coecke, B. (2019; revised 2020). The Mathematics of Text Structure. arXiv:1904.03478.

A direct precedent for representing text through circuits in DisCoCirc. Important for correctly attributing ideas and studying compositional updates to meanings.

[5] Cousot, P.; Cousot, R. (1977). Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. POPL 1977, pp. 238–252.

The foundational paper on abstract interpretation. A reference for the relationship between concrete states, approximate descriptions and fixed-point computations; its central guarantee must not be confused with exactness.

[6] Radul, A.; Sussman, G. J. (online technical report). Revised Report on the Propagator Model. MIT CSAIL; accessed September 11, 2026.

A computational model with cells accumulating partial information and components propagating consequences. Conceptually very close to wires carrying refinable knowledge.

[7] Goldreich, O. (2001). Foundations of Cryptography, Volume I: Basic Tools. Cambridge University Press; author's page.

A foundation for distinguishing forward computation, inversion and verification. One-way functions illustrate why knowing the algorithm does not automatically provide efficient search.

[8] Vaandrager, F.; Midya, A. (2020). A Myhill-Nerode Theorem for Register Automata and Symbolic Trace Languages. ICTAC 2020; arXiv:2007.03540, extended version available.

A reference for merging histories according to what continuations can distinguish. The extension to registers emphasizes that relations between values can form part of the relevant state.

[9] Shalizi, C. R.; Crutchfield, J. P. (1999; published 2001). Computational Mechanics: Pattern and Prediction, Structure and Simplicity. arXiv:cond-mat/9907176.

Studies process representations through the equivalence of future predictions. Useful for the connection between history, state and predictive sufficiency; it must not be confused with interventional causal identification.

[10] Darwiche, A.; Marquis, P. (2002). A Knowledge Compilation Map. Journal of Artificial Intelligence Research; arXiv copy:1106.1819.

Compares representation languages by succinctness, efficient queries and transformations. A reference for choosing representations according to use and accounting for compilation cost.

[11] Mateescu, R.; Dechter, R.; Marinescu, R. (2008). AND/OR Multi-Valued Decision Diagrams (AOMDDs) for Graphical Models. JAIR 33, pp. 465–519; arXiv copy:1401.3448.

Explains how to exploit independence and graphical decompositions. Relevant to frontiers, separators and the difference between the number of histories and the size of a shared representation.

[12] Darwiche, A. (2020). An Advance on Variable Elimination with Applications to Tensor-Based Computation. arXiv:2002.09320.

Shows the importance of functional dependencies for variable elimination. Supports investigating algebraic structure, rather than graph connectivity alone, as a source of efficiency.

[13] Polikarpova, N.; Kuraj, I.; Solar-Lezama, A. (2016). Program Synthesis from Polymorphic Refinement Types. PLDI 2016; preprint arXiv:1510.08419.

A precedent for synthesizing programs using types containing semantic conditions. Relevant to transforming a goal into component requirements.

[14] Guo, Z.; James, M.; Justo, D.; Zhou, J.; Wang, Z.; Jhala, R.; Polikarpova, N. (2020). Program Synthesis by Type-Guided Abstraction Refinement. POPL 2020; arXiv:1911.04091.

Presents TYGAR and the use of abstraction refinement in synthesis. Important to the idea of starting with a coarser description and introducing distinctions when needed.

[15] Alur, R. and colleagues (2013). Syntax-Guided Synthesis. FMCAD 2013; manuscript available on the author's page.

Separates the syntactic space of admissible programs from the behavioral specification. Provides a framework for explicitly discussing what an expander searches for and what the candidate evaluator checks.

[16] Clarke, E.; Grumberg, O.; Jha, S.; Lu, Y.; Veith, H. (2000). Counterexample-Guided Abstraction Refinement. CAV 2000, LNCS 1855, pp. 154–169.

The reference paper for CEGAR. A spurious counterexample may indicate a distinction lost in the abstract model, rather than an error in the concrete system.

[17] Willsey, M.; Nandi, C.; Wang, Y. R.; Flatt, O.; Tatlock, Z.; Panchekha, P. (2021). egg: Fast and Extensible Equality Saturation. POPL 2021; arXiv:2004.03082.

A reference for the shared representation of equivalent expressions. Validated equality must be distinguished from merely preserving alternative interpretations.

[18] Goodman, J. (1999). Semiring Parsing. Computational Linguistics 25(4), pp. 573–606.

Shows how a parsing structure can be evaluated using different algebras. Useful for distinguishing existence, counting, cost and weighting of derivations.

[19] Grädel, E.; Tannen, V. (2024). Provenance Analysis and Semiring Semantics for First-Order Logic. arXiv:2412.07986.

A framework for tracking logical dependencies through provenance semantics. Relevant to alternative justifications and the precautions required when negation comes into play.

[20] Soufflé Project (online documentation). Soufflé Tutorial. Official documentation; accessed September 11, 2026.

A practical introduction to Datalog relations and rules. An accessible entry point into the intuition of deriving facts, rather than a substitute for natural-language semantics.

[21] Vaswani, A. and colleagues (2017). Attention Is All You Need. NeurIPS 2017; arXiv:1706.03762.

A reference for the Transformer architecture. Cited here because distributed representations and neural operations can be described through explicit computational graphs.

[22] Yeung, C.; Zou, Z.; Jeong, S.; Huang, W.; Bastian, N. D.; Imani, M. (2024; revised version in 2026). Generalized Holographic Reduced Representations. arXiv:2405.09689.

Research on compositionality in distributed representations and binding operations. Offers a direction for comparison, rather than proof that holographic storage eliminates information loss.

[23] Ellis, K. and colleagues (2020). DreamCoder: Growing Generalizable, Interpretable Knowledge with Wake-Sleep Bayesian Program Learning. arXiv:2006.08381.

A precedent for learned symbolic libraries and neural exploration strategies. Relevant to separating program representation from the mechanism searching for programs.

[24] Tishby, N.; Pereira, F. C.; Bialek, W. (1999; preprint 2000). The Information Bottleneck Method. arXiv:physics/0004057.

Formalizes compression that preserves information relevant to a target. Helps discuss the trade-off between memory and prediction, without universal guarantees covering every future question.

[25] Strouse, D. J.; Schwab, D. J. (2016). The Deterministic Information Bottleneck. arXiv:1604.00268.

Explores an alternative formulation of information compression. The term deterministic must not be confused with eliminating uncertainty from the world or with a proof of semantic equivalence.

[26] Grünwald, P. (2004). A Tutorial Introduction to the Minimum Description Length Principle. arXiv:math/0406077.

A foundation for jointly evaluating model complexity and the data still to be described. Relevant to the cost of libraries and exceptions, rather than automatic identification of truth.

[27] Pearl, J. (2009). Causal Inference in Statistics: An Overview. Statistics Surveys 3, pp. 96–146; report available on the author's page.

Clarifies the role of causal models and the difference between observation and intervention. Important for avoiding confusion between a computation's dependencies and the world's causal mechanisms.

[28] Tafjord, O.; Dalvi Mishra, B.; Clark, P. (2021). ProofWriter: Generating Implications, Proofs, and Abductive Statements over Natural Language. Preprint arXiv:2012.13048, submitted in 2020.

A reference for evaluating conclusions alongside proofs in bounded linguistic tasks. It must not be extrapolated to the full competence of understanding real documents.

[29] Saha, S.; Ghosh, S.; Srivastava, S.; Bansal, M. (2020). PRover: Proof Generation for Interpretable Reasoning over Rules. arXiv:2010.02830.

A paper on generating proofs for rule-based inferences. Relevant to measuring correct answers and valid justifications separately.

[30] Koehler, T.; Trinder, P.; Steuwer, M. (2021). Sketch-Guided Equality Saturation: Scaling Equality Saturation to Complex Optimizations of Functional Programs. arXiv:2111.13040.

Studies guidance of a shared rewrite space. A useful warning that using an e-graph does not automatically remove the problem of search-space size.

[31] Nandi, C. and colleagues (2021). Rewrite Rule Inference Using Equality Saturation. arXiv:2108.10436.

A precedent for discovering useful equality rules. Relevant to learning circuits and transformations, provided the semantics of proposed rules are verified.

[32] Dannert, K. M.; Grädel, E.; Naaf, M.; Tannen, V. (2019). Generalized Absorptive Polynomials and Provenance Semantics for Fixed-Point Logic. arXiv:1910.07910.

Extends provenance analysis to fixed-point logics. Shows why the choice of justification algebra must be adapted to recursion and the logical properties pursued.